Northern University of Bussiness and Technology Khulna

NUBTK $(^w^)$

Sourav Mondal, Ayon Adhikary Arko, Devjyoti Tikader Dipto

Comming Soon

2026-06-17

Contest (1)

739397.cpp

18 lines

```
#include <bits/stdc++.h>
using namespace std;

template<class T> using V = vector<T>;
typedef long long ll;
typedef pair<int, int> pii;

#define f first
#define s second
#define sz(x) (int)(x).size()
#define all(x) begin(x), end(x)
#define each(a, x) for(auto& a : x)
#define FOR(i, a, b) for(int i = (a); i < (b); ++i)

int main() {
  cin.sync_with_stdio(0); cin.tie(0);
  cin.exceptions(cin.failbit);
}
```

.bashrc

3 lines

```
alias clr="printf '\33c'"
co() { g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -Wconversion -o $1 $1.cpp; }
run() { co $1 && ./$1; }
```

hash.sh

3 lines

```
# Hash file ignoring whitespace and comments. Verifies that
# code was correctly typed. Usage: 'sh hash.sh < A.cpp'
cpp -dD -P -fpreprocessed|tr -d '[:space:]'|md5sum|cut -c-6
```

gen.cpp

176 lines

```
#include "bits/stdc++.h"
#include "assert.h"
using namespace std;
//////////////// DISTRIBUTIONS
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

// return int in [L,R] inclusive
int rng_int(int L, int R) { assert(L <= R);
  return uniform_int_distribution<int>(L, R)(rng);
}
int64_t rng_int64(int64_t L, int64_t R) { assert(L <= R);
  return uniform_int_distribution<int64_t>(L, R)(rng);
}

// return double in [L,R] inclusive
double rng_db(double L, double R) { assert(L <= R);
  return uniform_real_distribution<double>(L, R)(rng);
}

// http://cplusplus.com/reference/random/geometric_distribution/geometric_distribution/
// flip a coin which is heads with probability p until you flip heads
// mean value of c is 1/p-1
int rng_geo(double p) { assert(0 < p && p <= 1); // p large -> closer to 0
  return geometric_distribution<int>(p)(rng);
}

//////////////// VECTORS + PERMS

// shuffle a vector
template<class T> void randShuffle(vector<T>& v) {
  shuffle(begin(v), end(v), rng);
}

// generate random permutation of [0,N-1]
vector<int> randPermutation(int N) {
  vector<int> v(N); iota(begin(v), end(v), 0);
  randShuffle(v); return v;
}

// random permutation of [0,N-1] with first element 0
vector<int> randPermutationZero(int N) {
  vector<int> v(N-1); iota(begin(v), end(v), 1);
  randShuffle(v);
  v.insert(begin(v), 0); return v;
}

// shuffle permutation of [0,N-1]
vi shufPerm(vi v) {
  int N = sz(v); vi key = randPerm(N);
  vi res(N); F0R(i,N) res[key[i]] = key[v[i]];
  return res;
}

// vector with all entries in [L,R]
vector<int> randVector(int N, int L, int R) {
  vector<int> v(N);
  for (int i = 0; i < N; i++) {
    v[i] = rng_int(L, R);
  }
  return v;
}

// vector with all entries in [L,R], unique
vector<int> rngDistinctVector(int N, int L, int R) {
  set<int> used; vector<int> v;
  while ((int)size(v) < N) {
    int x = rng_int(L, R);
    if (!used.count(x)) {
      used.insert(x);
      v.push_back(x);
    }
  } return v;
}

// generate random prime in [L,R]
int randPrime(int L, int R) { assert(L <= R && L >= 2);
  while(1) {
    int x = rng_int(L, R);
    bool bad = 0;
    for (int i = 2; i*i <= x; i++) if (x%i == 0) bad = 1;
    if (!bad) return x;
  }
}

//////////////// GRAPHS

// relabel edges ed according to perm, shuffle
vpi relabelAndShuffle(vpi ed, vi perm) {
  each(t,ed) {
    t.f = perm[t.f], t.s = perm[t.s];
    if (rng()&1) swap(t.f,t.s);
  }
  shuf(ed); return ed;
}

// shuffle graph with vertices [0,N-1]
vpi shufGraph(int N, vpi ed) { // randomly swap endpoints, rearrange labels
  return relabelAndShuffle(ed,randPerm(N)); }
vpi shufGraphZero(int N, vpi ed) {
  return relabelAndShuffle(ed,randPermZero(N)); }

// shuffle tree given N-1 edges
vpi shufTree(vpi ed) { return shufGraph(sz(ed)+1,ed); }
// randomly swap endpoints, rearrange labels
vpi shufRootedTree(vpi ed) {
  return relabelAndShuffle(ed,randPermZero(sz(ed)+1)); }

void pgraphOne(int N, vpi ed) {
  ps(N,sz(ed));
  each(e,ed) ps(1+e.f,1+e.s);
}

//////////////// GENERATING TREES

// for generating tall tree
pi geoEdge(int i, db p) { assert(i > 0);
  return {i,max(0,i-1-rng_geo(p))}; }

// generate edges of tree with verts [0,N-1]
// smaller back -> taller tree
vpi treeRand(int N, int back) {
  assert(N >= 1 && back >= 0); vpi ed;
  FOR(i,1,N) ed.eb(i,i-1-rng_int(0,min(back,i-1)));
  return ed; }

// generate path
vpi path(int N) { return treeRand(N,0); }

// generate tall tree (large diameter)
// the higher the p the taller the tree
vpi treeTall(int N, db p) { assert(N >= 1);
  vpi ed; FOR(i,1,N) ed.pb(geoEdge(i,p));
  return ed; }

// generate tall tree, then add rand at end
vpi treeTallShort(int N, db p) {
```

```
  assert(N >= 1); int mid = (N+1)/2;
  vpi ed = treeTall(mid,p);
  FOR(i,mid,N) ed.eb(i,rng_int(0,i-1));
  return ed; }

// lots of stuff connected to either heavy1 or heavy2
vpi treeTallHeavy(int N, db p) {
  assert(N >= 1); // + bunch of rand
  vpi ed; int heavy1 = 0, heavy2 = N/2;
  FOR(i,1,N) {
    if(i < N/4) ed.eb(i,heavy1);
    else if (i > heavy2 && i < 3*N/4) ed.eb(i,heavy2);
    else ed.pb(geoEdge(i,p));
  }
  return ed;
}

// heavy tall tree + random
// lots of verts connected to heavy1 or heavy2
vpi treeTallHeavyShort(int N, db p) {
  assert(N >= 1); // + almost-path + rand
  vpi ed; int heavy1 = 0, heavy2 = N/2;
  FOR(i,1,N) {
    if(i < N/4) ed.eb(i,heavy1);
    else if (i <= heavy2) ed.pb(geoEdge(i,p)); // tall -> heavy1
    else if (i > heavy2 && i < 3*N/4) ed.eb(i,heavy2);
    else ed.eb(i,rng_int(0,i-1));
  }
  return ed;
}

int main(int, char* argv[]) {
  int tc; sscanf(argv[1],"%d",&tc); // test case #
  rng.seed(tc);
  // sscanf(argv[2],"%d",&b); // random seed
  // sscanf(argv[3],"%d",&c); // also random seed

}
```

stress.sh

10 lines

```
# A and B are executables you want to compare, gen takes int
# as command line arg. Usage: 'sh stress.sh'
for((i = 1; ; ++i)); do
  echo $i
  ./gen $i > in
  ./sol < in > outSol
  ./slow < in > outSlow
  diff -w outSol outSlow || break
  # diff -w <(./A < in) <(./B < in) || break
done
```

troubleshoot.txt

75 lines

```
General:
Write down most of your thoughts, even if you're not sure
whether they're useful.
Give your variables (and files) meaningful names.
Stay organized and don't leave papers all over the place!
You should know what your code is doing ...

Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Remove debug output.
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output as well.
Read the full problem statement again.
Have you understood the problem correctly?
Are you sure your algorithm works?
Try writing a slow (but correct) solution.
Can your algorithm handle the whole range of input?
Did you consider corner cases (ex. n=1)?
Is your output format correct? (including whitespace)
Are you clearing all data structures between test cases?
Any uninitialized variables?
Any undefined behavior (array out of bounds)?
Any overflows or NaNs (or shifting ll by >=64 bits)?
Confusing N and M, i and j, etc.?
Confusing ++i and i++?
Return vs continue vs break?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some test cases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Rewrite your solution from the start or let a teammate do it.

Geometry:
Work with ints if possible.
Correctly account for numbers close to (but not) zero. Related:
for functions like acos make sure absolute val of input is not
(slightly) greater than one.
Correctly deal with vertices that are collinear, concyclic,
coplanar (in 3D), etc.
Subtracting a point from every other (but not itself)?

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What's your complexity? Large TL does not mean that something
simple (like NlogN) isn't intended.
Are you copying a lot of unnecessary data? (References)
Avoid vector, map. (use arrays/unordered_map)
How big is the input and output? (consider FastIO)
What do your teammates think about your algorithm?
Calling count() on multiset?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
If using pointers try BumpAllocator.
```

Mathematics (2)

2.1 Trigonometry

$$\sin(v+w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v+w) = \cos v \cos w - \sin v \sin w$$
$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2+b^2}, \phi = \operatorname{atan2}(b,a)$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $s = \dfrac{a+b+c}{2}$

Area: $A = \sqrt{s(s-a)(s-b)(s-c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\dfrac{\sin\alpha}{a} = \dfrac{\sin\beta}{b} = \dfrac{\sin\gamma}{c} = \dfrac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.3 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x = 1 + \tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x \sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) \qquad \int x e^{ax} dx = \frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.4 Sums

$$r^a + r^{a+1} + \cdots + r^b = \frac{r^{b+1} - r^a}{r-1}, r \neq 1$$

$$1 + r + r^2 + \cdots + r^\infty = \frac{1}{1-r}, \quad |r| < 1$$

$$a+(a+d)r+(a+2d)r^2+(a+3d)r^3+\cdots = \frac{a}{1-r}+\frac{dr}{(1-r)^2}, \quad |r| < 1$$

$$a + (a+d) + (a+2d) + \cdots + (a+(n-1)d) = \frac{n}{2}(2a+(n-1)d)$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1 + 3 + 5 + \cdots + 2n - 1 = n^2$$

$$2 + 4 + 6 + \cdots + 2n = n(n+1)$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.5 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.6 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation.

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

If X, Y are independent,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

Expectation is linear If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Data Structures (3)

3.1 STL

MapComparator.h

Description: example of function object (functor) for map or set
Usage: `set<int,cmp> s; map<int,int,cmp> m;`

5bfa6c, 1 lines

```
struct cmp{bool operator()(int l,int r)const{return l>r;}};
```

HashMap.h

Description: Hash map with similar API as unordered_map. Initial capacity must be a power of 2 if provided.
Usage: `gp_hash_table<int, int, chash> h({},{},{},{},{1<<16});`
Memory: ~1.5x unordered map
Time: ~3x faster than unordered map

<ext/pb_ds/assoc_container.hpp> e58314, 12 lines

```
using namespace __gnu_pbds;
struct chash {
  const uint64_t C = 4e18*acos(0)+71;
  const int R = chrono::steady_clock::now().time_since_epoch().
    ↪count();
  size_t operator()(uint64_t x) const {
    return __builtin_bswap64((x^R)*C);
  }
};
template <class K, class V> using HT = gp_hash_table<K, V,
  ↪chash>;
template <class K, class V> V get(HT<K, V> &u, K x) {
  auto it = u.find(x); return it == u.end() ? 0 : it->second;
}
```

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. Change null_type to get a map.
Time: $\mathcal{O}(\log N)$

<ext/pb_ds/assoc_container.hpp> 222518, 18 lines

```
using namespace __gnu_pbds;
template<class T> using IndexedSet = tree<T, null_type,
less<T>, rb_tree_tag, tree_order_statistics_node_update>;

#define ook order_of_key // position of the given element
#define fbo find_by_order // return an iterator

// Number of element <= k
int atMost(IndexedSet<pair<int, int>>& T, int k) {
  return T.order_of_key({k, INT_MAX});
}
// Number of element grater >= k
int atLeast(IndexedSet<pair<int, int>>& T, int k) {
  return T.size() - T.order_of_key({k, INT_MIN});
}
int getSum(IndexedSet<pair<int, int>>& T, int l, int r) {
  return atMost(T,r)-atMost(T,l-1);
}
```

SWAG.h

Description: SWAG (Sliding Window Aggregation) using two stacks. Supports queue operations with O(1) amortized aggregate queries. Works for any associative operation (OR, AND, XOR, GCD, MIN, MAX, SUM, etc).
Usage: `Queue<int> q;`
`q.push(x);`
`q.pop();`
`q.query(); // aggregate over all elements in queue`
Memory: $\mathcal{O}(n)$
Time: push $\mathcal{O}(1)$, pop $\mathcal{O}(1)$ amortized, query $\mathcal{O}(1)$

5aa600, 29 lines

```
template<class T> struct Stack {
  vector<pair<T, T>> stk;
  T cmb(T x) {
  return stk.empty() ? x : min(stk.back().second, x);
  }
  void push(T x) { stk.push_back({x, cmb(x)}); }
  void pop() { stk.pop_back(); }
  T agg() const { return stk.back().second; }
};

template<class T> struct Queue {
  Stack<T> in, out;
  T cmb(T a, T b) {
  return min(a, b);
  }
  void push(T x) { in.push(x); }
  void pop() {
    if (out.stk.empty()) while (!in.stk.empty()) {
      out.push(in.stk.back().first);
      in.pop();
    }
    out.pop();
  }
  T query() {
  if (in.stk.empty()) return out.agg();
  if (out.stk.empty()) return in.agg();
  return cmb(in.agg(), out.agg());
  }
};
```

3.2 1D Range Queries

SparseTable.h

Description: Static 1D range (min/max/gcd/lcm/or/and) query. If TL is an issue, use arrays instead of vectors and store values instead of indices.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(1)$

471d4b, 19 lines

```
template<class T> struct SparseTable {
  int N, LOG;
  vector<vector<T>> jmp;
  T cmb(T a, T b) { return min(a, b); }
  SparseTable(const vector<T>& v) {
    N = v.size(); LOG = __lg(N);
    jmp.resize(N, vector<T>(LOG + 1));
    for (int i = 0; i < N; ++i) jmp[i][0] = v[i];
    for (int j = 1; j <= LOG; ++j) {
      for (int i = 0; i + (1<<j) <= N; ++i) {
        jmp[i][j] = cmb(jmp[i][j-1], jmp[i+(1<<(j-1))][j-1]);
      }
```

```
    }
  }
  T query(int l, int r) {
    int d = __lg(r-l+1);
    return cmb(jmp[l][d], jmp[r-(1<<d)+1][d]);
  }
};
```

BIT.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value. 0-Indexed.
Time: Both operations are $\mathcal{O}(\log N)$.

29f859, 26 lines

```
template<class T> struct BIT {
  int N; vector<T> bit;
  BIT(int _N) {
    N = _N;
    bit.resize(N);
  }
  void add(int p, T x) {
    for (++p; p <= N; p += p&-p) bit[p - 1] += x;
  }
  T sum(int l, int r) { return sum(r+1)-sum(l); }
  T sum(int r) {
    T res = 0;
    for (; r; r -= r&-r) res += bit[r - 1];
    return res;
  }
  int lower_bound(T sum) {
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      int npos = pos + pw;
      if (npos <= N && bit[npos - 1] < sum)
        pos = npos, sum -= bit[pos - 1];
    }
    return pos;
  }
};
```

SegmentTree.h

Description: 1D point update and range query where `cmb` is any associative operation. `seg[1]==query(0,N-1)`.
Time: $\mathcal{O}(\log N)$

a09d9c, 36 lines

```
template<class T> struct SegmentTree {
  const T ID{};
  T cmb(T a, T b) { return a + b; }
  int N = 1; vector<T> seg;
  SegmentTree(int _N) {
    while (N < _N) N *= 2;
    seg.assign(2*N, ID);
  }
  void pull(int p) { seg[p] = cmb(seg[2*p],seg[2*p+1]); }
  void update(int p, T val) { // set val at position p
    seg[p += N] = val;
    for (p /= 2; p > 0; p /= 2) pull(p);
  }
  T query(int l, int r) { // zero-indexed, inclusive
    T lf = ID, rf = ID;
    for (l += N, r += N + 1; l < r; l /= 2, r /= 2) {
      if (l & 1) lf = cmb(lf, seg[l++]);
      if (r & 1) rf = cmb(seg[--r], rf);
    }
    return cmb(lf, rf);
  }
  // int first_at_least(int lo, int val, int ind, int l, int r)
    ↪ { // if seg stores max across range
  //  if (r < lo || val > seg[ind]) return -1;
  //  if (l == r) return l;
  //  int m = (l+r)/2;
  //  int res = first_at_least(lo,val,2*ind,l,m); if (res !=
    ↪-1) return res;
  //  return first_at_least(lo,val,2*ind+1,m+1,r);
  // }
  // int k_th_one(int k, int ind, int l, int r) {
  //  if (l == r) return l;
  //  int m = (l + r) / 2;
  //  int lc = 2*ind, rc = 2*ind+1;
  //  if (seg[lc] >= k) return k_th_one(k, lc, l, m);
  //  else return k_th_one(k-seg[lc], rc, m+1, r);
  // }
};
```

LazySegmentTree.h

Description: 1D range increment and sum query.
Usage: `LazySeg<int64_t, 1<<20> T; T.update(l, r, val);`
Time: $\mathcal{O}(\log N)$

e0742f, 42 lines

```
template<class T, int SZ>struct LazySeg {
  // SZ must be power of 2
  static_assert(__builtin_popcount(SZ) == 1);
  const T ID{};
  T cmb(T a, T b) { return a + b; }
  T seg[2*SZ], lazy[2*SZ];
  LazySeg() {
    for (int i = 0; i < 2*SZ; ++i) seg[i] = lazy[i] = ID;
  }
  // modify values for current node
  void push(int ind, int L, int R) {
    seg[ind] += (R-L+1) * lazy[ind]; // dependent on operation
    if (L != R) for (int i = 0; i < 2; ++i) lazy[2*ind+i] +=
      ↪lazy[ind];
    lazy[ind] = 0;
  }
  void pull(int ind) {
    seg[ind] = cmb(seg[2*ind], seg[2*ind+1]);
  }
  void build() {
    for (int i = SZ-1; i >= 1; --i) pull(i);
  }
  void update(int lo, int hi, T inc, int ind = 1, int L = 0,
    ↪int R = SZ-1) {
    push(ind, L, R);
    if (hi < L || R < lo) return;
    if (lo <= L && R <= hi) {
      lazy[ind] = inc;
      push(ind, L, R);
      return;
    }
    int M = (L+R) / 2;
    update(lo, hi, inc, 2*ind, L, M);
    update(lo, hi, inc, 2*ind+1, M+1, R);
    pull(ind);
  }
  T query(int lo, int hi, int ind = 1, int L = 0, int R = SZ-1)
    ↪ {
    push(ind, L, R);
    if (lo > R || L > hi) return ID;
    if (lo <= L && R <= hi) return seg[ind];
    int M = (L+R) / 2;
    return cmb(query(lo, hi, 2*ind, L, M), query(lo, hi, 2*ind
      ↪+1, M+1, R));
  }
};
```

3.3 2D Range Queries

PrefixSum2D.h

Description: calculates rectangle sums in constant time

65b070, 17 lines

```
template<typename T> struct PrefixSum2D {
  vector<vector<T>> sum;
  PrefixSum2D(const vector<vector<T>> &v) {
    int n = v.size(), m = v[0].size();
    sum.assign(n+1, vector<T>(m+1, T{}));
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) {
        sum[i+1][j+1] = v[i][j]
        - sum[i][j] + sum[i+1][j] + sum[i][j+1];
      }
    }
  }
  T query(int x1, int y1, int x2, int y2) {
    return sum[x2][y2] + sum[x1-1][y1-1]
    - sum[x1-1][y2] - sum[x2][y1-1];
  }
};
```

Number Theory (4)

4.1 Modular Arithmetic

ModIntShort.h

Description: Modular arithmetic. Assumes MOD is prime.
Usage: `mint a = MOD+5; inv(a); // 400000003`

7c1e94, 30 lines

```
struct mint {
  int v;
  static const int MOD = 1E9 + 7;
  explicit operator int() const { return v; }
  mint() : v(0) {}
  mint(int64_t _v) : v(int(_v % MOD)) { v += (v < 0) * MOD; }
  mint &operator+=(mint o) {
    if ((v += o.v) >= MOD) v -= MOD;
    return *this;
  }
  mint &operator-=(mint o) {
    if ((v -= o.v) < 0) v += MOD;
    return *this;
  }
  mint &operator*=(mint o) {
    v = int((int64_t)v * o.v % MOD);
    return *this;
  }
  friend mint pow(mint a, int64_t p) {
    assert(p >= 0);
    return p==0? 1:pow(a*a, p/2)*(p&1? a:1);
  }
  friend mint inv(mint a) {
    assert(a.v != 0);
    return pow(a, MOD - 2);
  }
  friend mint operator+(mint a, mint b) { return a += b; }
  friend mint operator-(mint a, mint b) { return a -= b; }
  friend mint operator*(mint a, mint b) { return a *= b; }
};
```

ModFact.h

Description: Combinations modulo a prime MOD. Assumes $2 \leq N \leq MOD$.
Usage: `F.init(10); F.C(6, 4); // 15`
Time: $\mathcal{O}(N)$

"ModInt.h" 364271, 13 lines

```
struct {
```

```
  vmi invs, fac, ifac;
  void init(int N) { // idempotent
    invs.rsz(N), fac.rsz(N), ifac.rsz(N);
    invs[1] = fac[0] = ifac[0] = 1;
    FOR(i,2,N) invs[i] = mi(-(ll)MOD/i*(int)invs[MOD%i]);
    FOR(i,1,N) fac[i] = fac[i-1]*i, ifac[i] = ifac[i-1]*invs[i
      ↪];
  }
  mi C(int a, int b) {
    if (a < b || b < 0) return 0;
    return fac[a]*ifac[b]*ifac[a-b];
  }
} F;
```

ModMulLL.h

Description: Multiply two 64-bit integers mod another if 128-bit is not available. modMul is equivalent to `(ul)(__int128(a)*b%mod)`. Works for $0 \le a, b < mod < 2^{63}$. NOTE : Only works if system supports 80-bit floating point.

d22eee, 10 lines

```
using ul = uint64_t;
ul modMul(ul a, ul b, const ul mod){
  return __int128(a) * __int128(b) % mod;
}
ul modPow(ul a, ul b, const ul mod) {
  if (b == 0) return 1;
  ul res = modPow(a, b/2, mod);
  res = modMul(res, res, mod);
  return b&1 ? modMul(res, a, mod) : res;
}
```

FastMod.h

Description: Barrett reduction computes $a\%b$ about 4 times faster than usual where $b > 1$ is constant but not known at compile time. Division by b is replaced by multiplication by m and shifting right 64 bits.

ce43ec, 9 lines

```
using ul = uint64_t;
struct FastMod {
  ul b, m;
  FastMod(ul b) : b(b), m(-1ULL / b) {}
  ul reduce(ul a) {
    ul q = (ul)((__uint128_t(m) * a) >> 64), r = a - q*b;
    return r - (r>=b) * b;
  }
};
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for square roots mod a prime. -1 if doesn't exist.

Usage: `sqrt(mi((ll)1e10)); // 100000`

Time: $\mathcal{O}\left(\log^2(MOD)\right)$

"ModInt.h" bcfa63, 14 lines

```
using T = int;
T sqrt(mi a) {
  mi p = pow(a,(MOD-1)/2);
  if (p.v != 1) return p.v == 0 ? 0 : -1;
  T s = MOD-1; int r = 0; while (s%2 == 0) s /= 2, ++r;
  mi n = 2; while (pow(n,(MOD-1)/2).v == 1) n = T(n)+1;
  // n non-square, ord(g)=2^r, ord(b)=2^m, ord(g)=2^r, m<r
  for (mi x = pow(a,(s+1)/2), b = pow(a,s), g = pow(n,s);;) {
    if (b.v == 1) return min(x.v,MOD-x.v); // x^2=ab
    int m = 0; for (mi t = b; t.v != 1; t *= t) ++m;
    rep(r-m-1) g *= g; // ord(g)=2^{m+1}
    x *= g, g *= g, b *= g, r = m; // ord(g)=2^m, ord(b)<2^m
  }
}
```

ModSum.h

Description: Counts # of lattice points (x, y) in the triangle $1 \le x, 1 \le y, ax + by \le s \pmod{2^{64}}$ and related quantities.

Time: $\mathcal{O}(\log ab)$

23cbf6, 20 lines

```
using ul = uint64_t;
ul sum2(ul n) { return n/2*((n-1)|1); } // sum(0..n-1)
// \return |{(x,y) | 1 <= x, 1 <= y, a*x+b*y <= S}|
//         = sum_{i=1}^{qs} (S-a*i)/b
ul triSum(ul a, ul b, ul s) { assert(a > 0 && b > 0);
  ul qs = s/a, rs = s%a; // ans = sum_{i=0}^{qs-1}(i*a+rs)/b
  ul ad = a/b*sum2(qs)+rs/b*qs; a %= b, rs %= b;
  return ad+(a?triSum(b,a,a*qs+rs):0); // reduce if a >= b
} // then swap x and y axes and recurse

// \return sum_{x=0}^{n-1} (a*x+b)/m
//         = |{(x,y) | 0 < m*y <= a*x+b < a*n+b}|
// assuming a*n+b does not overflow
ul divSum(ul n, ul a, ul b, ul m) { assert(m > 0);
  ul extra = b/m*n; b %= m;
  return extra+(a?triSum(m,a,a*n+b):0); }
// \return sum_{x=0}^{n-1} (a*x+b)%m
ul modSum(ul n, ll a, ll b, ul m) { assert(m > 0);
  a = (a%m+m)%m, b = (b%m+m)%m;
  return a*sum2(n)+b*n-m*divSum(n,a,b,m); }
```

4.2 Primality

4.2.1 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^{\times}$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

4.2.2 Divisors

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

Let $n = \prod_i p_i^{e_i}$ be the prime factorization of n.

Number of Divisors: $d(n) = \prod_i (e_i + 1)$.

Sum of Divisors: $\sigma(n) = \prod_i \frac{p_i^{e_i+1}-1}{p_i-1}$.

Product of Divisors:

Case 1: n is a perfect square. Then $\prod_{d|n} d = \sqrt{n}^{d(n)}$.

Case 2: n is not a perfect square. Then $\prod_{d|n} d = n^{d(n)/2}$.

Euler's Totient Function:
$\phi(n) = n \prod_i (1 - \frac{1}{p_i}) = \prod_i (p_i^{a_i} - p_i^{a_i-1})$.

Dirichlet Convolution: Given a function $f(x)$, let

$(f * g)(x) = \sum_{d|x} g(d) f(x/d)$.

If the partial sums $s_{f*g}(n), s_g(n)$ can be computed in $O(1)$ and $s_f(1 \dots n^{2/3})$ can be computed in $O\left(n^{2/3}\right)$ then all $s_f\left(\frac{n}{d}\right)$ can as well. Use

$$s_{f*g}(n) = \sum_{d=1}^{n} g(d) s_f(n/d).$$

If $f(x) = \mu(x)$ then $g(x) = 1, (f * g)(x) = (x == 1)$, and $s_f(n) = 1 - \sum_{i=2}^{n} s_f(n/i)$.

If $f(x) = \phi(x)$ then $g(x) = 1, (f * g)(x) = x$, and $s_f(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} s_f(n/i)$.

Let $s(x) = \sum_{i=1}^{x} \phi(i)$. Then

$$s(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} s\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

can be computed in faster than $\Theta(n)$.

FactorBasic.h

Description: Factors integers.

Time: $\mathcal{O}\left(\sqrt{N}\right)$

8b0341, 50 lines

```
inline namespace factorBasic {
template<class T> vector<pair<T, int>> factor(T x) {
  vector<pair<T, int>> primes;
  for (T i = 2; i*i <= x; ++i) if (x%i == 0) {
    int t = 0;
    while (x%i == 0) x /= i, t++;
    primes.push_back({i, t});
  }
  if (x > 1) primes.push_back({x, 1});
  return primes;
}

/* Note:
 * number of operations needed s.t.
 *          phi(phi(...phi(n)...))=1
 * is O(log n).
 * Euler's theorem: a^{\phi(p)}\equiv 1 (mod p), gcd(a,p)=1
 */
template<class T> T phi(T x) {
  for (auto& [p, exp]: factor(x)) x -= x/p;
  return x;
}

template<class T> void tour(vector<pair<T, int>> &v, vector<T>
  ↪&V, int ind, T cur) {
  if (ind == (int)size(v)) V.push_back(cur);
  else {
    T mul = 1;
    for (int i = 0; i <= v[ind].second; i++) {
      tour(v, V, ind+1, cur*mul);
      mul *= v[ind].first;
    }
  }
}

template<class T> vector<T> getDiv(T x) {
  auto v = factor(x);
  vector<T> V;
  tour(v, V, 0, (T)1);
```

```
  ranges::sort(V); return V;
}

template<class T> vector<T> getDiv(T x) {
  vector<T> V;
  for (T i = 1; i*i <= x; ++i) if (x%i == 0) {
    V.push_back(i);
    if (x/i != i) V.push_back(x/i);
  }
  ranges::sort(V); return V;
}
}
```

Sieve.h

Description: Tests primality up to N. Runs faster if only odd indices are stored.
Time: $\mathcal{O}(N \log\log N)$ or $\mathcal{O}(N)$

c10506, 27 lines

```
template<int N> struct Sieve {
  bitset<N> is_prime; vector<int> primes;
  Sieve() {
    is_prime.set(); is_prime[0] = is_prime[1] = 0;
    for (int i = 4; i < N; i += 2) is_prime[i] = 0;
    for (int i = 3; i*i < N; i += 2) if (is_prime[i])
      for (int j = i*i; j < N; j += i*2) is_prime[j] = 0;
    for (int i = 0; i < N; i++) if (is_prime[i]) primes.
      ↪push_back(i);
  }
  int spf[N];
  Sieve() {
    for (int i = 1; i < N; i += 2) spf[i] = i;
    for (int i = 2; i < N; i += 2) spf[i] = 2;
    for (int i = 3; i*i < N; i += 2) if (spf[i] == i)
    for (int j = i*i; j < N; j += i*2) spf[j] == j ? spf[j] = i
      ↪;
  }
  // int spf[N];
  // Sieve() { // above is faster
  //  for (int i = 2; i < N; i++) {
  //    if (spf[i] == 0) spf[i] = i, primes.push_back(i);
  //    for (int p: primes) {
  //      if (p > spf[i] || i*p >= N) break;
  //      spf[i*p] = p;
  //    }
  //  }
  // }
};
```

ETF.h

Description: Computes Euler Totient function
Time: $\mathcal{O}(N \log\log N)$

7549e9, 11 lines

```
template<int N> struct ETF {
  int phi[N];
  ETF() {
    for (int i = 0; i < N; i++) phi[i] = i;
    for (int i = 2; i < N; i++) if (phi[i] == i) {
      for (int j = i; j < N; j += i) {
        phi[j] -= phi[j] / i;
      }
    }
  }
};
```

MultiplicativePrefixSums.h

Description: $\sum_{i=1}^{N} f(i)$ where $f(i) = \prod \texttt{val}[e]$ for each p^e in the factorization of i. Must satisfy $\texttt{val}[1] = 1$. Generalizes to any multiplicative function with $f(p) = p^{\text{fixed power}}$.
Time: $\mathcal{O}\left(\sqrt{N}\right)$

"Sieve.h" 3151ea, 12 lines

```
vmi val;
mi get_prefix(ll N, int p = 0) {
  mi ans = N;
  for (; S.primes.at(p) <= N / S.primes.at(p); ++p) {
    ll new_N = N / S.primes.at(p) / S.primes.at(p);
    for (int idx = 2; new_N; ++idx, new_N /= S.primes.at(p)) {
      ans += (val.at(idx) - val.at(idx - 1))
           * get_prefix(new_N, p + 1);
    }
  }
  return ans;
}
```

PrimeCnt.h

Description: Counts number of primes up to N. Can also count sum of primes.
Time: $\mathcal{O}\left(N^{3/4}/\log N\right)$, 60ms for $N = 10^{11}$, 2.5s for $N = 10^{13}$

c04e96, 20 lines

```
ll count_primes(ll N) { // count_primes(1e13) == 346065536839
  if (N <= 1) return 0;
  int sq = (int)sqrt(N);
  vl big_ans((sq+1)/2), small_ans(sq+1);
  FOR(i,1,sq+1) small_ans[i] = (i-1)/2;
  F0R(i,sz(big_ans)) big_ans[i] = (N/(2*i+1)-1)/2;
  vb skip(sq+1); int prime_cnt = 0;
  for (int p = 3; p <= sq; p += 2) if (!skip[p]) { // primes
    for (int j = p; j <= sq; j += 2*p) skip[j] = 1;
    F0R(j,min((ll)sz(big_ans),(N/p/p+1)/2)) {
      ll prod = (ll)(2*j+1)*p;
      big_ans[j] -= (prod > sq ? small_ans[(double)N/prod]
             : big_ans[prod/2])-prime_cnt;
    }
    for (int j = sq, q = sq/p; q >= p; --q) for (;j >= q*p;--j)
      small_ans[j] -= small_ans[q]-prime_cnt;
    ++prime_cnt;
  }
  return big_ans[0]+1;
}
```

MillerRabin.h

Description: Deterministic primality test, works up to 2^{64}. For larger numbers, extend A randomly.

"ModMulLL.h" f5b0fe, 11 lines

```
bool prime(ul n) { // not ll!
  if (n < 2 || n % 6 % 4 != 1) return n - 2 < 2;
  ul s = __builtin_ctzll(n - 1), d = n >> s;
  ul A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
  for (auto a : A) {
    ul p = modPow(a, d, n), i = s;
    while (p != 1 && p != n-1 && a % n && i--) p = modMul(p, p,
      ↪ n);
    if (p != n-1 && i != s) return false;
  }
  return true;
}
```

FactorFast.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(N^{1/4}\right)$, less for numbers with small factors

"MillerRabin.h", "ModMulLL.h" 99cf33, 16 lines

```
ul pollard(ul n) { // return some nontrivial factor of n
  auto f = [n](ul x) { return modMul(x, x, n) + 1; };
  ul x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  while (t++ % 40 || gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modMul(prd, max(x,y)-min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return gcd(prd, n);
}
void factor_rec(ul n, map<ul,int>& cnt) {
  if (n == 1) return;
  if (prime(n)) { ++cnt[n]; return; }
  ul u = pollard(n);
  factor_rec(u,cnt), factor_rec(n/u,cnt);
}
```

4.3 Euclidean Algorithm

4.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

FracInterval.h

Description: Given fractions $a < b$ with non-negative numerators and denominators, finds fraction f with lowest denominator such that $a < f < b$. Should work with all numbers less than 2^{62}.

1860f3, 6 lines

```
pl bet(pl a, pl b) {
  ll num = a.f/a.s; a.f -= num*a.s, b.f -= num*b.s;
  if (b.f > b.s) return {1+num,1};
  auto x = bet({b.s,b.f},{a.s,a.f});
  return {x.s+num*x.f,x.f};
}
```

Euclid.h

Description: Generalized Euclidean algorithm. `euclid` and `invGeneral` work for $A, B < 2^{62}$.
Time: $\mathcal{O}(\log AB)$

8bab06, 13 lines

```
// For A,B>=0, finds (x,y) s.t.
pair<int64_t, int64_t> euclid(int64_t A, int64_t B) {
  // Ax+By=gcd(A,B), |Ax|,|By|<=AB/gcd(A,B)
  if (!B) return {1, 0};
  pair<int64_t, int64_t> p = euclid(B, A%B);
  return {p.second, p.first - A/B*p.second};
}
// find x in [0,B) such that Ax=1 mod B
int64_t invGeneral(int64_t A, int64_t B) {
  pair<int64_t, int64_t> p = euclid(A, B);
  assert(p.first*A + p.second*B == 1);
  return p.first + (p.first<0)*B;
} // must have gcd(A,B)=1
```

CRT.h

Description: Chinese Remainder Theorem. $a.f \pmod{a.s}, b.f \pmod{b.s} \Longrightarrow\ ? \pmod{\operatorname{lcm}(a.s, b.s)}$. Should work for $ab < 2^{62}$.

"Euclid.h" 23df64, 10 lines

```
pl CRT(pl a, pl b) { assert(0 <= a.f && a.f < a.s && 0 <= b.f
   ↪&& b.f < b.s);
  if (a.s < b.s) swap(a,b); // will overflow if b.s^2 > 2^{62}
  ll x,y; tie(x,y) = euclid(a.s,b.s);
  ll g = a.s*x+b.s*y, l = a.s/g*b.s;
```

```
  if ((b.f-a.f)%g) return {-1,-1}; // no solution
  // ?*a.s+a.f \equiv b.f \pmod{b.s}
  // ?=(b.f-a.f)/g*(a.s/g)^{-1} \pmod{b.s/g}
  x = (b.f-a.f)%b.s*x%b.s/g*a.s+a.f;
  return {x+(x<0)*l,l};
}
```

ModArith.h

Description: Statistics on mod'ed arithmetic series. `minBetween` and `minRemainder` both assume that $0 \le L \le R < B$, $AB < 2^{62}$

f68a6d, 40 lines

```
ll minBetween(ll A, ll B, ll L, ll R) {
  // min x s.t. exists y s.t. L <= A*x-B*y <= R
  A %= B;
  if (L == 0) return 0;
  if (A == 0) return -1;
  ll k = cdiv(L,A); if (A*k <= R) return k;
  ll x = minBetween(B,A,A-R%A,A-L%A); // min x s.t. exists y
  // s.t. -R <= Bx-Ay <= -L
  return x == -1 ? x : cdiv(B*x+L,A); // solve for y
}

// find min((Ax+C)%B) for 0 <= x <= M
// aka find minimum non-negative value of A*x-B*y+C
// where 0 <= x <= M, 0 <= y
ll minRemainder(ll A, ll B, ll C, ll M) {
  assert(A >= 0 && B > 0 && C >= 0 && M >= 0);
  A %= B, C %= B; ckmin(M,B-1);
  if (A == 0) return C;
  if (C >= A) { // make sure C<A
    ll ad = cdiv(B-C,A);
    M -= ad; if (M < 0) return C;
    C += ad*A-B;
  }
  ll q = B/A, new_B = B%A; // new_B < A
  if (new_B == 0) return C; // B-q*A

  // now minimize A*x-new_B*y+C
  // where 0 <= x,y and x+q*y <= M, 0 <= C < new_B < A
  // q*y -> C-new_B*y
  if (C/new_B > M/q) return C-M/q*new_B;
  M -= C/new_B*q; C %= new_B; // now C < new_B

  // given y, we can compute x = ceil[((B-q*A)*y-C)/A]
  // so x+q*y = ceil((B*y-C)/A) <= M
  ll max_Y = (M*A+C)/B; // must have y <= max_Y
  ll max_X = cdiv(new_B*max_Y-C,A); // must have x <= max_X
  if (max_X*A-new_B*max_Y+C >= new_B) --max_X;
  // now we can remove upper bound on y
  return minRemainder(A,new_B,C,max_X);
}
```

4.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k\cdot(m^2-n^2),\ \ b = k\cdot(2mn),\ \ c = k\cdot(m^2+n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

4.5 Lifting the Exponent

For $n > 0$, p prime, and ints x, y s.t. $p \nmid x, y$ and $p | x - y$:

- $p \neq 2$ or $p = 2, 4|x-y \implies v_p(x^n - y^n) = v_p(x-y) + v_p(n)$.
- $p = 2, 2|n \implies v_2(x^n - y^n) = v_2((x^2)^{n/2} - (y^2)^{n/2})$.

Combinatorial (5)

5.1 Permutations

5.1.1 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

5.1.2 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

5.2 Partitions and subsets

5.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ \ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

5.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

5.3 General purpose numbers

5.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able). $B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

5.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ \ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

5.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \ge j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

5.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j} j^n$$

5.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

5.3.6 Labeled unrooted trees

\# on n vertices: n^{n-2}
\# on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
\# with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

5.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

5.4 Young Tableaux

Let a **Young diagram** have shape $\lambda = (\lambda_1 \geq \cdots \geq \lambda_k)$, where λ_i equals the number of cells in the i-th (left-justified) row from the top. A **Young tableau** of shape λ is a filling of the $n = \sum \lambda_i$ cells with a permutation of $1 \ldots n$ such that each row and column is increasing.

Hook-Length Formula: For the cell in position (i,j), let $h_\lambda(i,j) = |\{(I,J) | i \leq I, j \leq J, (I = i \text{ or } J = j)\}|$. The number of Young tableaux of shape λ is equal to $f^\lambda = \frac{n!}{\prod h_\lambda(i,j)}$.

Schensted's Algorithm: converts a permutation σ of length n into a pair of Young Tableaux $(S(\sigma), T(\sigma))$ of the same shape. When inserting $x = \sigma_i$,

1. Add x to the first row of S by inserting x in place of the largest y with $x < y$. If y doesn't exist, push x to the end of the row, set the value of T at that position to be i, and stop.

2. Add y to the second row using the same rule, keep repeating as necessary.

All pairs $(S(\sigma), T(\sigma))$ of the same shape correspond to a unique σ, so $n! = \sum (f^\lambda)^2$. Also, $S(\sigma^R) = S(\sigma)^T$.

Let $d_k(\sigma), a_k(\sigma)$ be the lengths of the longest subseqs which are a union of k decreasing/ascending subseqs, respectively. Then $a_k(\sigma) = \sum_{i=1}^k \lambda_i, d_k(\sigma) = \sum_{i=1}^k \lambda_i^*$, where λ_i^* is size of the i-th column.

5.5 Other

DeBruijnSequence.h

Description: Given alphabet $[0, k)$ constructs a cyclic string of length k^n that contains every length n string as substring.

dfccad, 14 lines

```
vector<int> deBruijnSequence(int k, int n) {
  if (k == 1) return {0};
  vector<int> seq, aux(n+1);
  auto gen = [&](this auto&& self, int t, int p)->void {
    if (t > n && n%p == 0) for (int i = 1; i <= p; i++) {
      seq.push_back(aux[i]);
    }
    else if (t <= n) {
      aux[t] = aux[t-p]; self(t+1, p);
      while (++aux[t] < k) self(t+1, t);
    }
  };
  gen(1, 1); return seq;
}
```

NimProduct.h

Description: Product of nimbers is associative, commutative, and distributive over addition (xor). Forms finite field of size 2^{2^k}. Defined by $ab = \text{mex}(\{a'b + ab' + a'b' : a' < a, b' < b\})$. Application: Given 1D coin turning games G_1, G_2 $G_1 \times G_2$ is the 2D coin turning game defined as follows. If turning coins at $x_1, x_2, \ldots, x_m$ is legal in G_1 and $y_1, y_2, \ldots, y_n$ is legal in G_2, then turning coins at all positions (x_i, y_j) is legal assuming that the coin at (x_m, y_n) goes from heads to tails. Then the grundy function $g(x,y)$ of $G_1 \times G_2$ is $g_1(x) \times g_2(y)$.

Time: 64^2 xors per multiplication, memorize to speed up.

5afe17, 46 lines

```
using ul = uint64_t;
struct Precalc {
  ul tmp[64][64], y[8][8][256];
  unsigned char x[256][256];
  Precalc() { // small nim products, all < 256
    F0R(i,256) F0R(j,256) x[i][j] = mult<8>(i,j);
    F0R(i,8) F0R(j,i+1) F0R(k,256)
      y[i][j][k] = mult<64>(prod2(8*i,8*j),k);
  }
  ul prod2(int i, int j) { // nim prod of 2^i, 2^j
    ul& u = tmp[i][j]; if (u) return u;
    if (!(i&j)) return u = 1ULL<<(i|j);
    int a = (i&j)&-(i&j); // a=2^k, consider 2^{2^k}
    return u=prod2(i^a,j)^prod2((i^a)|(a-1),(j^a)|(i&(a-1)));
    // 2^{2^k}*2^{2^k} = 2^{2^k}+2^{2^k-1}
  } // 2^{2^i}*2^{2^j} = 2^{2^i+2^j} if i<j
  template<int L> ul mult(ul a, ul b) {
    ul c = 0; F0R(i,L) if (a>>i&1)
      F0R(j,L) if (b>>j&1) c ^= prod2(i,j);
    return c;
  }
  // 2^{8*i}*(a>>(8*i)&255) * 2^{8*j}*(b>>(8*j)&255)
  // -> (2^{8*i}*2^{8*j})*((a>>(8*i)&255)*(b>>(8*j)&255))
  ul multFast(ul a, ul b) const { // faster nim product
    ul res = 0; auto f=[](ul c,int d) {return c>>(8*d)&255;};
    F0R(i,8) {
      F0R(j,i) res ^= y[i][j][x[f(a,i)][f(b,j)]
                ^x[f(a,j)][f(b,i)]];
      res ^= y[i][i][x[f(a,i)][f(b,i)]];
    }
    return res;
  }
};
const Precalc P;

struct nb { // nimber
  ul x; nb() { x = 0; }
  nb(ul _x): x(_x) {}
  explicit operator ul() { return x; }
  nb operator+(nb y) { return nb(x^y.x); }
  nb operator*(nb y) { return nb(P.multFast(x,y.x)); }
  friend nb pow(nb b, ul p) {
    nb res = 1; for (;p;p/=2,b=b*b) if (p&1) res = res*b;
    return res; } // b^{2^{2^A}-1}=1 where 2^{2^A} > b
  friend nb inv(nb b) { return pow(b,-2); }
};
```

MatroidIsect.h

Description: Computes a set of maximum size which is independent in both graphic and colorful matroids, aka a spanning forest where no two edges are of the same color. In general, construct the exchange graph and find a shortest path. Can apply similar concept to partition matroid.

Usage: `MatroidIsect<Gmat,Cmat> M(sz(ed),Gmat(ed),Cmat(col))`

Time: $\mathcal{O}\left(GI^{1.5}\right)$ calls to oracles, where G is size of ground set and I is size of independent set.

"DSU.h" d0051c, 51 lines

```
struct Gmat { // graphic matroid
  int V = 0; vpi ed; DSU D;
  Gmat(vpi _ed):ed(_ed) {
    map<int,int> m; each(t,ed) m[t.f] = m[t.s] = 0;
    each(t,m) t.s = V++;
    each(t,ed) t.f = m[t.f], t.s = m[t.s];
  }
  void clear() { D.init(V); }
  void ins(int i) { assert(D.unite(ed[i].f,ed[i].s)); }
  bool indep(int i) { return !D.sameSet(ed[i].f,ed[i].s); }
};
struct Cmat { // colorful matroid
  int C = 0; vi col; V<bool> used;
  Cmat(vi col):col(col) {each(t,col) ckmax(C,t+1); }
  void clear() { used.assign(C,0); }
  void ins(int i) { used[col[i]] = 1; }
  bool indep(int i) { return !used[col[i]]; }
};
template<class M1, class M2> struct MatroidIsect {
  int n; V<bool> iset; M1 m1; M2 m2;
  bool augment() {
    vi pre(n+1,-1); queue<int> q({n});
    while (sz(q)) {
      int x = q.ft; q.pop();
      if (iset[x]) {
        m1.clear(); F0R(i,n) if (iset[i] && i != x) m1.ins(i);
        F0R(i,n) if (!iset[i] && pre[i] == -1 && m1.indep(i))
          pre[i] = x, q.push(i);
      } else {
        auto backE = [&]() { // back edge
          m2.clear();
          F0R(c,2)F0R(i,n)if((x==i||iset[i])&&(pre[i]==-1)==c){
            if (!m2.indep(i))return c?pre[i]=x,q.push(i),i:-1;
            m2.ins(i); }
          return n;
        };
        for (int y; (y = backE()) != -1;) if (y == n) {
          for(; x != n; x = pre[x]) iset[x] = !iset[x];
          return 1; }
      }
    }
    return 0;
  }
  MatroidIsect(int n, M1 m1, M2 m2):n(n), m1(m1), m2(m2) {
    iset.assign(n+1,0); iset[n] = 1;
    m1.clear(); m2.clear(); // greedily add to basis
    R0F(i,n) if (m1.indep(i) && m2.indep(i))
      iset[i] = 1, m1.ins(i), m2.ins(i);
    while (augment());
  }
};
```

Numerical (6)

6.1 Matrix

Matrix.h

Description: 2D matrix operations.

"ModInt.h" 45a081, 57 lines

```
using T = mi;
using Mat = vector<vector<T>>; // use array instead if tight TL

Mat makeMat(int r, int c) {
  return Mat(r, vector<T>(c));
}
Mat makeId(int n) {
  Mat m = makeMat(n, n);
  for (int i = 0; i < n; i++) m[i][i] = 1;
  return m;
}

Mat operator+(Mat& a, const Mat& b) {
  assert(sz(a) == sz(b) && sz(a[0]) == sz(b[0]));
  int x = size(a), y = size(a[0]);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < y; j++) {
      a[i][j] += b[i][j];
    }
  }
  return a;
}

Mat operator-(Mat& a, const Mat& b) {
  assert(sz(a) == sz(b) && sz(a[0]) == sz(b[0]));
  int x = size(a), y = size(a[0]);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < y; j++) {
      a[i][j] -= b[i][j];
    }
  }
  return a;
}

Mat operator*(const Mat& a, const Mat& b) {
  int x = size(a), y = size(a[0]), z = size(b[0]);
  assert(y == sz(b)); Mat c = makeMat(x, z);
  for (int i = 0; i < x; i++) {
    for (int j = 0; j < y; j++) {
      for (int k = 0; k < z; k++) {
        c[i][k] += a[i][j]*b[j][k];
      }
    }
  }
  return c;
}

Mat& operator*=(Mat& a, const Mat& b) { return a = a*b; }
Mat& operator+=(Mat& a, const Mat& b) { return a = a+b; }
Mat& operator-=(Mat& a, const Mat& b) { return a = a-b; }

Mat pow(Mat m, int64_t p) {
  int n = sz(m); assert(n == sz(m[0]) && p >= 0);
  Mat res = makeId(n);
  for (; p; p /= 2, m *= m) if (p&1) res *= m;
  return res;
}
```

MatrixInv.h

Description: Uses gaussian elimination to convert into reduced row echelon form and calculates determinant. For determinant via arbitrary modulos, use a modified form of the Euclidean algorithm because modular inverse may not exist. If you have computed $A^{-1} \pmod{p^k}$, then the inverse $\pmod{p^{2k}}$ is $A^{-1}(2I - AA^{-1})$.
Time: $\mathcal{O}\left(N^3\right)$, determinant of 1000×1000 matrix of modints in 1 second if you reduce # of operations by half

73ec43, 38 lines

```
const db EPS = 1e-9; // adjust?
int getRow(V<V<db>>& m, int R, int i, int nex) {
  pair<db,int> bes{0,-1}; // find row with max abs value
  FOR(j,nex,R) ckmax(bes,{abs(m[j][i]),j});
  return bes.f < EPS ? -1 : bes.s; }
int getRow(V<vmi>& m, int R, int i, int nex) {
  FOR(j,nex,R) if (m[j][i] != 0) return j;
  return -1; }
pair<T,int> gauss(Mat& m) { // convert to reduced row echelon
  ↪form
  if (!sz(m)) return {1,0};
  int R = sz(m), C = sz(m[0]), rank = 0, nex = 0;
  T prod = 1; // determinant
  F0R(i,C) {
    int row = getRow(m,R,i,nex);
    if (row == -1) { prod = 0; continue; }
    if (row != nex) prod *= -1, swap(m[row],m[nex]);
    prod *= m[nex][i]; rank++;
    T x = 1/m[nex][i]; FOR(k,i,C) m[nex][k] *= x;
    F0R(j,R) if (j != nex) {
      T v = m[j][i]; if (v == 0) continue;
      FOR(k,i,C) m[j][k] -= v*m[nex][k];
    }
    nex++;
  }
  return {prod,rank};
}
Mat inv(Mat m) {
  int R = sz(m); assert(R == sz(m[0]));
  Mat x = makeMat(R,2*R);
  F0R(i,R) {
    x[i][i+R] = 1;
    F0R(j,R) x[i][j] = m[i][j];
  }
  if (gauss(x).s != R) return Mat();
  Mat res = makeMat(R,R);
  F0R(i,R) F0R(j,R) res[i][j] = x[i][j+R];
  return res;
}
```

MatrixTree.h

Description: Kirchhoff's Matrix Tree Theorem. Given adjacency matrix, calculates # of spanning trees.

"MatrixInv.h" 48363d, 11 lines

```
T numSpan(const Mat& m) {
  int n = sz(m); Mat res = makeMat(n-1,n-1);
  F0R(i,n) FOR(j,i+1,n) {
    mi ed = m[i][j]; res[i][i] += ed;
    if (j != n-1) {
      res[j][j] += ed;
      res[i][j] -= ed, res[j][i] -= ed;
    }
  }
  return gauss(res).f;
}
```

ShermanMorrison.h

Description: Calculates $(A + uv^T)^{-1}$ given $B = A^{-1}$. Not invertible if sum=0.

"MatrixInv.h" 3a3f34, 7 lines

```
void ad(Mat& B, const V<T>& u, const V<T>& v) {
  int n = sz(A); V<T> x(n), y(n);
  F0R(i,n) F0R(j,n)
    x[i] += B[i][j]*u[j], y[j] += v[i]*B[i][j];
  T sum = 1; F0R(i,n) F0R(j,n) sum += v[i]*B[i][j]*u[j];
  F0R(i,n) F0R(j,n) B[i][j] -= x[i]*y[j]/sum;
}
```

6.2 Polynomials

Poly.h

Description: Basic poly ops including division. Can replace T with double, complex.

"ModInt.h" cd218a, 73 lines

```
using T = mi; using poly = V<T>;
void remz(poly& p) { while (sz(p)&&p.bk==T(0)) p.pop_back(); }
poly REMZ(poly p) { remz(p); return p; }
poly rev(poly p) { reverse(all(p)); return p; }
poly shift(poly p, int x) {
  if (x >= 0) p.insert(begin(p),x,0);
  else assert(sz(p)+x >= 0), p.erase(begin(p),begin(p)-x);
  return p;
}
poly RSZ(const poly& p, int x) {
  if (x <= sz(p)) return poly(begin(p),begin(p)+x);
  poly q = p; q.rsz(x); return q;  }
T eval(const poly& p, T x) { // evaluate at point x
  T res = 0; R0F(i,sz(p)) res = x*res+p[i];
  return res; }
poly dif(const poly& p) { // differentiate
  poly res; FOR(i,1,sz(p)) res.pb(T(i)*p[i]);
  return res; }
poly integ(const poly& p) { // integrate
  static poly invs{0,1};
  for (int i = sz(invs); i <= sz(p); ++i)
    invs.pb(-MOD/i*invs[MOD%i]);
  poly res(sz(p)+1); F0R(i,sz(p)) res[i+1] = p[i]*invs[i+1];
  return res;
}

poly& operator+=(poly& l, const poly& r) {
  l.rsz(max(sz(l),sz(r))); F0R(i,sz(r)) l[i] += r[i];
  return l; }
poly& operator-=(poly& l, const poly& r) {
  l.rsz(max(sz(l),sz(r))); F0R(i,sz(r)) l[i] -= r[i];
  return l; }
poly& operator*=(poly& l, const T& r) { each(t,l) t *= r;
  return l;  }
poly& operator/=(poly& l, const T& r) { each(t,l) t /= r;
  return l; }
poly operator+(poly l, const poly& r) { return l += r; }
poly operator-(poly l, const poly& r) { return l -= r; }
poly operator-(poly l) { each(t,l) t *= -1; return l; }
poly operator*(poly l, const T& r) { return l *= r; }
poly operator*(const T& r, const poly& l) { return l*r; }
poly operator/(poly l, const T& r) { return l /= r;  }
poly operator*(const poly& l, const poly& r) {
  if (!min(sz(l),sz(r))) return {};
  poly x(sz(l)+sz(r)-1);
  F0R(i,sz(l)) F0R(j,sz(r)) x[i+j] += l[i]*r[j];
  return x;
}
poly& operator*=(poly& l, const poly& r) { return l = l*r; }

pair<poly,poly> quoRemSlow(poly a, poly b) {
  remz(a); remz(b); assert(sz(b));
  T lst = b.bk, B = T(1)/lst; each(t,a) t *= B;
  each(t,b) t *= B;
  poly q(max(sz(a)-sz(b)+1,0));
  for (int dif; (dif=sz(a)-sz(b)) >= 0; remz(a)) {
    q[dif] = a.bk; F0R(i,sz(b)) a[i+dif] -= q[dif]*b[i]; }
  each(t,a) t *= lst;
  return {q,a}; // quotient, remainder
}
poly operator%(const poly& a, const poly& b) {
  return quoRemSlow(a,b).s; }
T resultant(poly a, poly b) { // R(A,B)
```

```
  // =b_m^n*prod_{j=1}^mA(mu_j)
  // =b_m^na_n^m*prod_{i=1}^nprod_{j=1}^m(mu_j-lambda_i)
  // =(-1)^{mn}a_n^m*prod_{i=1}^nB(lambda_i)
  // =(-1)^{nm}R(B,A)
  // Also, R(A,B)=b_m^{deg(A)-deg(A-CB)}R(A-CB,B)
  int ad = sz(a)-1, bd = sz(b)-1;
  if (bd <= 0) return bd < 0 ? 0 : pow(b.bk,ad);
  int pw = ad; a = a%b; pw -= (ad = sz(a)-1);
  return resultant(b,a)*pow(b.bk,pw)*T((bd&ad&1)?-1:1);
}
```

PolyInterpolate.h

Description: n points determine unique polynomial of degree $\le n-1$. For numerical precision pick $v[k].f = c * \cos(k/(n-1)*\pi), k = 0 \dots n-1$.
Time: $\mathcal{O}\left(n^2\right)$

"Poly.h" aada3a, 8 lines

```
poly interpolate(V<pair<T,T>> v) {
  poly res, tmp{1};
  F0R(i,sz(v)) { T prod = 1; // add one point at a time
    F0R(j,i) v[i].s -= prod*v[j].s, prod *= v[i].f-v[j].f;
    v[i].s /= prod; res += v[i].s*tmp; tmp *= poly{-v[i].f,1};
  } // add multiple of (x-v[0].f)*(x-v[1].f)*...*(x-v[i-1].f)
  return res;
}
```

FFT.h

Description: Multiply polynomials of ints for any modulus $< 2^{31}$. For XOR convolution ignore `m` within `fft`.
Time: $\mathcal{O}(N \log N)$. For $N = 10^6$, conv ~0.13ms, conv_general ~320ms.

"ModInt.h" 19ab20, 39 lines

```
// const int MOD = 998244353;
tcT> void fft(V<T>& A, bool invert = 0) { // NTT
  int n = sz(A); assert((T::mod-1)%n == 0); V<T> B(n);
  for(int b = n/2; b; b /= 2, swap(A,B)) { // w = n/b'th root
    T w = pow(T::rt(),(T::mod-1)/n*b), m = 1;
    for(int i = 0; i < n; i += b*2, m *= w) F0R(j,b) {
      T u = A[i+j], v = A[i+j+b]*m;
      B[i/2+j] = u+v; B[i/2+j+n/2] = u-v;
    }
  }
  if (invert) { reverse(1+all(A));
    T z = inv(T(n)); each(t,A) t *= z; }
} // for NTT-able moduli
tcT> V<T> conv(V<T> A, V<T> B) {
  if (!min(sz(A),sz(B))) return {};
  int s = sz(A)+sz(B)-1, n = 1; for (; n < s; n *= 2);
  A.rsz(n), fft(A); B.rsz(n), fft(B);
  F0R(i,n) A[i] *= B[i];
  fft(A,1); A.rsz(s); return A;
}
template<class M, class T> V<M> mulMod(const V<T>& x, const V<T
  ↪>& y) {
  auto con = [](const V<T>& v) {
    V<M> w(sz(v)); F0R(i,sz(v)) w[i] = (int)v[i];
    return w; };
  return conv(con(x), con(y));
} // arbitrary moduli
tcT> V<T> conv_general(const V<T>& A, const V<T>& B) {
  using m0 = mint<(119<<23)+1,62>; auto c0 = mulMod<m0>(A,B);
  using m1 = mint<(5<<25)+1,  62>; auto c1 = mulMod<m1>(A,B);
  using m2 = mint<(7<<26)+1,  62>; auto c2 = mulMod<m2>(A,B);
  int n = sz(c0); V<T> res(n); m1 r01 = inv(m1(m0::mod));
  m2 r02 = inv(m2(m0::mod)), r12 = inv(m2(m1::mod));
  F0R(i,n) { // a=remainder mod m0::mod, b fixes it mod m1::mod
    int a = c0[i].v, b = ((c1[i]-a)*r01).v,
      c = (((c2[i]-a)*r02-b)*r12).v;
    res[i] = (T(c)*m1::mod+b)*m0::mod+a; // c fixes m2::mod
  }
  return res;
}
```

PolyInvSimpler.h

Description: computes A^{-1} such that $AA^{-1} \equiv 1 \pmod{x^n}$. Newton's method: If you want $F(x) = 0$ and $F(Q_k) \equiv 0 \pmod{x^a}$ then $Q_{k+1} = Q_k - \frac{F(Q_k)}{F'(Q_k)} \pmod{x^{2a}}$ satisfies $F(Q_{k+1}) \equiv 0 \pmod{x^{2a}}$. Application: if $f(n), g(n)$ are the #s of forests and trees on n nodes then $\sum_{n=0}^{\infty} f(n)x^n = \exp\left(\sum_{n=1}^{\infty} \frac{g(n)}{n!}\right)$.
Usage: `vmi v{1,5,2,3,4}; ps(exp(2*log(v,9),9)); // squares v`
Time: $\mathcal{O}(N \log N)$. For $N = 5 \cdot 10^5$, inv~270ms, log ~350ms, exp~550ms

"FFT.h", "Poly.h" 6e5362, 30 lines

```
poly inv(poly A, int n) { // Q-(1/Q-A)/(-Q^{-2})
  poly B{inv(A[0])};
  for (int x = 2; x/2 < n; x *= 2)
    B = 2*B-RSZ(conv(RSZ(A,x),conv(B,B)),x);
  return RSZ(B,n);
}
poly sqrt(const poly& A, int n) {  // Q-(Q^2-A)/(2Q)
  assert(A[0].v == 1); poly B{1};
  for (int x = 2; x/2 < n; x *= 2)
    B = inv(T(2))*RSZ(B+conv(RSZ(A,x),inv(B,x)),x);
  return RSZ(B,n);
}
// return {quotient, remainder}
pair<poly,poly> quoRem(const poly& f, const poly& g) {
  if (sz(f) < sz(g)) return {{},f};
  poly q = conv(inv(rev(g),sz(f)-sz(g)+1),rev(f));
  q = rev(RSZ(q,sz(f)-sz(g)+1));
  poly r = RSZ(f-conv(q,g),sz(g)-1); return {q,r};
}
poly log(poly A, int n) { assert(A[0].v == 1); // (ln A)' = A'/
  ↪A
  A.rsz(n); return integ(RSZ(conv(dif(A),inv(A,n-1)),n-1)); }
poly exp(poly A, int n) { assert(A[0].v == 0);
  poly B{1}, IB{1}; // inverse of B
  for (int x = 1; x < n; x *= 2) {
    IB = 2*IB-RSZ(conv(B,conv(IB,IB)),x);
    poly Q = dif(RSZ(A,x)); Q += RSZ(conv(IB,dif(B)-conv(B,Q))
      ↪,2*x-1);
    B = B+RSZ(conv(B,RSZ(A,2*x)-integ(Q)),2*x);
  }
  return RSZ(B,n);
}
```

6.3 Misc

LinearRecurrence.h

Description: Berlekamp-Massey. Computes linear recurrence `C` of order N for sequence `s` of $2N$ terms. $C[0] = 1$ and for all $i \ge sz(C) - 1$, $\sum_{j=0}^{sz(C)-1} C[j]s[i-j] = 0$.
Usage: `LinRec L; L.init({0,1,1,2,3}); L.eval(5); L.eval(6); // 5, 8`
Time: `init` $\Rightarrow \mathcal{O}(N|C|)$, `eval` $\Rightarrow \mathcal{O}\left(|C|^2 \log p\right)$ or faster with FFT

"Poly.h" 39ea71, 29 lines

```
struct LinRec {
  poly s, C, rC;
  void BM() {
    int x = 0; T b = 1;
    poly B; B = C = {1}; // B is fail vector
    F0R(i,sz(s)) { // update C after adding a term of s
      ++x; int L = sz(C), M = i+3-L;
      T d = 0; F0R(j,L) d += C[j]*s[i-j]; // [D^i]C*s
      if (d.v == 0) continue; // [D^i]C*s=0
      poly _C = C; T coef = d*inv(b);
      C.rsz(max(L,M)); F0R(j,sz(B)) C[j+x] -= coef*B[j];
      if (L < M) B = _C, b = d, x = 0;
    }
  }
  void init(const poly& _s) {
    s = _s; BM();
    rC = C; reverse(all(rC));
    C.erase(begin(C)); each(t,C) t *= -1;
  } // now s[i]=sum_{j=0}^{sz(C)-1}C[j]*s[i-j-1]
  poly getPow(ll p) { // get x^p mod rC
    if (p == 0) return {1};
    poly r = getPow(p/2); r = (r*r)%rC;
    return p&1?(r*poly{0,1})%rC:r;
  }
  T dot(poly v) { // dot product with s
    T ans = 0; F0R(i,sz(v)) ans += v[i]*s[i];
    return ans; } // get p-th term of rec
  T eval(ll p) { assert(p >= 0); return dot(getPow(p)); }
};
```

Integrate.h

Description: Integration of a function over an interval using Simpson's rule, exact for polynomials of degree up to 3. The error should be proportional to dif^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.
Usage: `quad([](db x) { return x*x+3*x+1; }, 2, 3) // 14.833...`

3ebaab, 6 lines

```
template<class F> db quad(F f, db a, db b) {
  const int n = 1000;
  db dif = (b-a)/2/n, tot = f(a)+f(b);
  FOR(i,1,2*n) tot += f(a+i*dif)*(i&1?4:2);
  return tot*dif/3;
}
```

IntegrateAdaptive.h

Description: Unused. Fast integration using adaptive Simpson's rule, exact for polynomials of degree up to 5.
Usage: `db z, y;`
```
db h(db x) { return x*x + y*y + z*z <= 1; }
db g(db y) { ::y = y; return quad(h, -1, 1); }
db f(db z) { ::z = z; return quad(g, -1, 1); }
db sphereVol = quad(f,-1,1), pi = sphereVol*3/4;
```

3b316e, 10 lines

```
template<class F> db simpson(F f, db a, db b) {
  db c = (a+b)/2; return (f(a)+4*f(c)+f(b))*(b-a)/6; }
template<class F> db rec(F& f, db a, db b, db eps, db S) {
  db c = (a+b)/2;
  db S1 = simpson(f,a,c), S2 = simpson(f,c,b), T = S1+S2;
  if (abs(T-S)<=15*eps || b-a<1e-10) return T+(T-S)/15;
  return rec(f,a,c,eps/2,S1)+rec(f,c,b,eps/2,S2);
}
template<class F> db quad(F f, db a, db b, db eps = 1e-8) {
  return rec(f,a,b,eps,simpson(f,a,b)); }
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns `-inf` if there is no solution, `inf` if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: `vvd A{{1,-1}, {-1,1}, {-1,-2}};`
```
vd b{1,1,-4}, c{-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
```
Time: $\mathcal{O}(NM \cdot \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}\left(2^N\right)$ in the general case.

c99f9c, 67 lines

```
using T = db; // double probably suffices
using vd = V<T>; using vvd = V<vd>;
const T eps = 1e-8, inf = 1/.0;
```

```
#define ltj(X) if (s==-1 || mp(X[j],N[j])<mp(X[s],N[s])) s=j
struct LPSolver {
  int m, n; // # m = contraints, # n = variables
  vi N, B; // N[j] = non-basic variable (j-th column), = 0
  vvd D; // B[j] = basic variable (j-th row)
  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
    F0R(i,m) F0R(j,n) D[i][j] = A[i][j];
    F0R(i,m) B[i] = n+i, D[i][n] = -1, D[i][n+1] = b[i];
    // B[i]: basic variable for each constraint
    // D[i][n]: artificial variable for testing feasibility
    F0R(j,n) N[j] = j, D[m][j] = -c[j];
    // D[m] stores negation of objective,
    // which we want to minimize
    N[n] = -1; D[m+1][n] = 1; // to find initial feasible
  } // solution, minimize artificial variable
  void pivot(int r, int s) { // swap B[r] (row)
    T inv = 1/D[r][s]; // with N[r] (column)
    F0R(i,m+2) if (i != r && abs(D[i][s]) > eps) {
      T binv = D[i][s]*inv;
      F0R(j,n+2) if (j != s) D[i][j] -= D[r][j]*binv;
      D[i][s] = -binv;
    }
    D[r][s] = 1; F0R(j,n+2) D[r][j] *= inv; // scale r-th row
    swap(B[r],N[s]);
  }
  bool simplex(int phase) {
    int x = m+phase-1;
    while (1) { // if phase=1, ignore artificial variable
      int s = -1; F0R(j,n+1) if (N[j] != -phase) ltj(D[x]);
      // find most negative col for nonbasic (NB) variable
      if (D[x][s] >= -eps) return 1;
      // can't get better sol by increasing NB variable
      int r = -1;
      F0R(i,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || mp(D[i][n+1] / D[i][s], B[i])
             < mp(D[r][n+1] / D[r][s], B[r])) r = i;
        // find smallest positive ratio
      } // -> max increase in NB variable
      if (r == -1) return 0; // objective is unbounded
      pivot(r,s);
    }
  }
  T solve(vd& x) { // 1. check if x=0 feasible
    int r = 0; FOR(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) { // if not, find feasible start
      pivot(r,n); // make artificial variable basic
      assert(simplex(2)); // I think this will always be true??
      if (D[m+1][n+1] < -eps) return -inf;
      // D[m+1][n+1] is max possible value of the negation of
      // artificial variable, optimal value should be zero
      // if exists feasible solution
      F0R(i,m) if (B[i] == -1) { // artificial var basic
        int s = 0; FOR(j,1,n+1) ltj(D[i]); // -> nonbasic
        pivot(i,s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    F0R(i,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

Graphs (7)

Erdos-Gallai: $d_1 \geq \cdots \geq d_n$ can be degree sequence of simple graph on n vertices iff their sum is even and $\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k), \forall 1 \leq k \leq n$.

7.1 Basics

DirectedCycle.h

Description: stack

c746ee, 28 lines

```
struct DirectedCycle {
  int N;
  vector<bool> inStk, vis;
  vector<int> stk, cyc;
  vector<vector<int>> adj;
  DirectedCycle(int _N) {
    N = _N;
    adj.resize(N);
    inStk.resize(N), vis.resize(N);
  }
  void ae(int u, int v) { adj[u].push_back(v); }
  vector<int> gen() {
    for (int u = 0; u < N; u++) {
      if (!vis[u] && empty(cyc)) dfs(u);
    }
    return cyc;
  }
  void dfs(int u) {
    stk.push_back(u);
    inStk[u] = vis[u] = 1;
    for (auto& v: adj[u]) {
      if (inStk[v]) cyc = {ranges::find(stk, v), end(stk)};
      else if (!vis[v]) dfs(v);
      if (size(cyc)) return;
    }
    stk.pop_back(); inStk[u] = 0;
  }
};
```

GridBFS.h

Description: BFS through grid with fixed xdir and ydir arrays

c9585f, 43 lines

```
using P = array<int, 2>;
tuple<int, int, char> moves[]{
  {0, 1, 'R'}, {1, 0, 'D'}, {0, -1, 'L'}, {-1, 0, 'U'}
};
struct GridBFS {
  int N, M;
  vector<vector<int>> dist;
  vector<string> grid, dir;
  vector<vector<P>> pre;
  GridBFS(int _N, int _M, vector<string>& _g) {
    N = _N, M = _M, grid = _g;
    dist.resize(N, vector(M, -1));
    dir.resize(N, string(M, '.'));
    pre.resize(N, vector<P>(M, {-1, -1}));
  }
  bool valid(int x, int y) {
    return !(x < 0 || x >= N || y < 0 || y >= M);
  }
  void gen(P& st) {
    queue<P> todo;
    todo.push(st); dist[st[0]][st[1]] = 0;
    while (todo.size()) {
      auto u = todo.front(); todo.pop();
      for (auto& [dx, dy, di]: moves) {
        P v = {u[0]+dx, u[1]+dy};
        if (!valid(v[0], v[1]) || dist[v[0]][v[1]] != -1)
          ↪continue;
        todo.push(v);
        dist[v[0]][v[1]] = dist[u[0]][u[1]] + 1;
        pre[v[0]][v[1]] = u, dir[v[0]][v[1]] = di;
      }
    }
  }
  string getPath(P& st, P en) {
    if (dist[en[0]][en[1]] == -1) return "";
    string res;
    while (en != st) {
      auto& [x, y] = en;
      res += dir[x][y]; en = pre[x][y];
    }
    ranges::reverse(res);
    return res;
  }
};
```

Bipartite.h

Description: Checks if an undirected graph is bipartite. Colors nodes with 1/2. If not bipartite, returns false.

Time: $\mathcal{O}(N+M)$

4f7998, 28 lines

```
struct Bipartite {
  int N;
  vector<int> color;
  vector<vector<int>> adj;
  Bipartite(int _N) {
    N = _N;
    adj.resize(N); color.resize(N);
  }
  void ae(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  bool check() {
    queue<int> todo;
    for (int src = 0; src < N; src++) {
      if (color[src]) continue;
      color[src] = 1; todo.push(src);
      while (!todo.empty()) {
        int u = todo.front(); todo.pop();
        for (auto &v: adj[u]) {
          if (color[v] == color[u]) return false;
          if (!color[v]) color[v] = 3-color[u], todo.push(v);
        }
      }
    }
    return true;
  }
};
```

TopoSort.h

Description: sorts vertices such that if there exists an edge u->v, then u goes before v

7b10d8, 20 lines

```
struct TopoSort {
  int N;
  vector<int> in, res;
  vector<vector<int>> adj;
  TopoSort(int _N) {
    N = _N; in.resize(N); adj.resize(N);
  }
  void ae(int u, int v) {
    adj[u].push_back(v), ++in[v];
  }
  bool sort() {
    queue<int> todo;
    for (int u = 0; u < N; ++u) if (!in[u]) todo.push(u);
    while (size(todo)) {
```

```
      int u = todo.front(); todo.pop(); res.push_back(u);
      for (auto &v: adj[u]) if (!(--in[v])) todo.push(v);
    }
    return (int)size(res) == N;
  }
};
```

Dijkstra.h

Description: shortest path
Time: $\mathcal{O}(N \log N + M)$

8cfac5, 27 lines

```
template<class T, bool directed> struct Dijkstra {
  int N;
  vector<T> dist;
  vector<vector<pair<int, T>>> adj;
  Dijkstra(int _N) {
    N = _N;
    adj.resize(N);
  }
  void ae(int u, int v, T w) {
    adj[u].push_back({v, w});
    if (!directed) adj[v].push_back({u, w});
  }
  void gen(int st) {
    dist.assign(N, numeric_limits<T>::max());
    priority_queue<pair<T, int>> PQ;
    auto relax = [&](int v, T d) {
      if (dist[v] <= d) return;
      dist[v] = d;
      PQ.push({-dist[v], v});
    }; relax(st, 0);
    while (!PQ.empty()) {
      auto [d, u] = PQ.top(); PQ.pop(); d = -d;
      if (dist[u] < d) continue;
      for (auto& [v, w]: adj[u]) relax(v, w+d);
    }
  }
};
```

NegativeCycle.h

Description: use Bellman-Ford (make sure no underflow)

e3180d, 22 lines

```
vector<int> NegativeCycle(int N, vector<array<int, 3>> &ed) {
  vector<int64_t> dist(N);
  vector<int> p(N);
  int x = -1;
  for (int i = 0; i < N; i++) {
    x = -1;
    for (auto &[u, v, w] : ed) {
      if (dist[v] > dist[u]+w) {
        dist[v] = dist[u]+w;
        p[v] = u, x = v;
      }
    }
    if (x == -1) return {};
  }

  for (int i = 0; i < N; i++) x = p[x]; // enter cycle
  vector<int> cyc{x};
  while (p[cyc.back()] != x) cyc.push_back(p[cyc.back()]);
  cyc.push_back(x); // complete cycle
  ranges::reverse(cyc);
  return cyc;
}
```

BellmanFord.h

Description: Shortest Path w/ negative edge weights

c2a89c, 33 lines

```
struct BellmanFord {
  int N;
  vector<int64_t> dist;
  vector<vector<int>> adj;
  vector<array<int, 3>> ed;
  BellmanFord(int _N) {
    N = _N;
    adj.resize(N);
    dist.resize(N, INF);
  }
  void ae(int u, int v, int w) {
    adj[u].push_back(v);
    ed.push_back({u, v, w});
  }
  void genBad(int u) {
  if (dist[u] == -INF) return;
    dist[u] = -INF;
    for (auto &v : adj[u]) genBad(v);
  }
  void genDist(int src) {
    dist[src] = 0;
    for (int i = 0; i < N; i++) {
      for (auto &[u, v, w] : ed) {
        if (dist[u] < INF) {
          dist[v] = min(dist[v], dist[u]+w);
        }
      }
    }
    for (auto &[u, v, w] : ed) {
      if (dist[u] < INF && dist[v] > dist[u]+w) genBad(v);
    }
  }
};
```

7.2 Trees

LCAjump.h

Description: Calculates least common ancestor in tree with verts $0 \ldots N-1$ and root R using binary jumping.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(\log N)$ query

6b0ee9, 28 lines

```
struct LCA {
  int N; V<vi> par, adj; vi depth;
  void init(int _N) {  N = _N;
    int d = 1; while ((1<<d) < N) ++d;
    par.assign(d,vi(N)); adj.rsz(N); depth.rsz(N);
  }
  void ae(int x, int y) { adj[x].pb(y), adj[y].pb(x); }
  void gen(int R = 0) { par[0][R] = R; dfs(R); }
  void dfs(int x = 0) {
    FOR(i,1,sz(par)) par[i][x] = par[i-1][par[i-1][x]];
    each(y,adj[x]) if (y != par[0][x])
      depth[y] = depth[par[0][y]=x]+1, dfs(y);
  }
  int jmp(int x, int d) {
    F0R(i,sz(par)) if ((d>>i)&1) x = par[i][x];
    return x; }
  int lca(int x, int y) {
    if (depth[x] < depth[y]) swap(x,y);
    x = jmp(x,depth[x]-depth[y]); if (x == y) return x;
    R0F(i,sz(par)) {
      int X = par[i][x], Y = par[i][y];
      if (X != Y) x = X, y = Y;
    }
    return par[0][x];
  }
  int dist(int x, int y) { // # edges on path
    return depth[x]+depth[y]-2*depth[lca(x,y)]; }
};
```

LCArmq.h

Description: Euler Tour LCA. Compress takes a subset S of nodes and computes the minimal subtree that contains all the nodes pairwise LCAs and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(1)$ LCA, $\mathcal{O}(|S| \log |S|)$ compress

"RMQ.h" e5a035, 28 lines

```
struct LCA {
  int N; V<vi> adj;
  vi depth, pos, par, rev; // rev is for compress
  vpi tmp; RMQ<pi> r;
  void init(int _N) { N = _N; adj.rsz(N);
    depth = pos = par = rev = vi(N); }
  void ae(int x, int y) { adj[x].pb(y), adj[y].pb(x); }
  void dfs(int x) {
    pos[x] = sz(tmp); tmp.eb(depth[x],x);
    each(y,adj[x]) if (y != par[x]) {
      depth[y] = depth[par[y]=x]+1, dfs(y);
      tmp.eb(depth[x],x); }
  }
  void gen(int R = 0) { par[R] = R; dfs(R); r.init(tmp); }
  int lca(int u, int v){
    u = pos[u], v = pos[v]; if (u > v) swap(u,v);
    return r.query(u,v).s; }
  int dist(int u, int v) {
    return depth[u]+depth[v]-2*depth[lca(u,v)]; }
  vpi compress(vi S) {
    auto cmp = [&](int a, int b) { return pos[a] < pos[b]; };
    sort(all(S),cmp); R0F(i,sz(S)-1) S.pb(lca(S[i],S[i+1]));
    sort(all(S),cmp); S.erase(unique(all(S)),end(S));
    vpi ret{{0,S[0]}}; F0R(i,sz(S)) rev[S[i]] = i;
    FOR(i,1,sz(S)) ret.eb(rev[lca(S[i-1],S[i])],S[i]);
    return ret;
  }
};
```

HLD.h

Description: Heavy-Light Decomposition, add val to verts and query sum in path/subtree.
Time: any tree path is split into $\mathcal{O}(\log N)$ parts

"LazySeg.h" 1802e2, 48 lines

```
template<int SZ, bool VALS_IN_EDGES> struct HLD {
  int N; vi adj[SZ];
  int par[SZ], root[SZ], depth[SZ], sz[SZ], ti;
  int pos[SZ]; vi rpos; // rpos not used but could be useful
  void ae(int x, int y) { adj[x].pb(y), adj[y].pb(x); }
  void dfsSz(int x) {
    sz[x] = 1;
    each(y,adj[x]) {
      par[y] = x; depth[y] = depth[x]+1;
      adj[y].erase(find(all(adj[y]),x));
      dfsSz(y); sz[x] += sz[y];
      if (sz[y] > sz[adj[x][0]]) swap(y,adj[x][0]);
    }
  }
  void dfsHld(int x) {
    pos[x] = ti++; rpos.pb(x);
    each(y,adj[x]) {
      root[y] = (y == adj[x][0] ? root[x] : y);
      dfsHld(y); }
  }
  void init(int _N, int R = 0) { N = _N;
    par[R] = depth[R] = ti = 0; dfsSz(R);
    root[R] = R; dfsHld(R);
  }
  int lca(int x, int y) {
    for (; root[x] != root[y]; y = par[root[y]])
      if (depth[root[x]] > depth[root[y]]) swap(x,y);
```

```
    return depth[x] < depth[y] ? x : y;
  }
  LazySeg<ll,SZ> tree; // segtree for sum
  template <class BinaryOp>
  void processPath(int x, int y, BinaryOp op) {
    for (; root[x] != root[y]; y = par[root[y]]) {
      if (depth[root[x]] > depth[root[y]]) swap(x,y);
      op(pos[root[y]],pos[y]); }
    if (depth[x] > depth[y]) swap(x,y);
    op(pos[x]+VALS_IN_EDGES,pos[y]);
  }
  void modifyPath(int x, int y, int v) {
    processPath(x,y,[this,&v](int l, int r) {
      tree.upd(l,r,v); }); }
  ll queryPath(int x, int y) {
    ll res = 0; processPath(x,y,[this,&res](int l, int r) {
      res += tree.query(l,r); });
    return res; }
  void modifySubtree(int x, int v) {
    tree.upd(pos[x]+VALS_IN_EDGES,pos[x]+sz[x]-1,v); }
};
```

Centroid.h

Description: The centroid of a tree of size N is a vertex such that after removing it, all resulting subtrees have size at most $\frac{N}{2}$. Supports updates in the form "add 1 to all verts v such that $dist(x, v) \leq y$."
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(\log N)$ update and query

907e21, 54 lines

```
void ad(vi& a, int b) { ckmin(b,sz(a)-1); if (b>=0) a[b]++; }
void prop(vi& a) { R0F(i,sz(a)-1) a[i] += a[i+1]; }
template<int SZ> struct Centroid {
  vi adj[SZ]; void ae(int a,int b){adj[a].pb(b),adj[b].pb(a);}
  bool done[SZ]; // processed as centroid yet
  int N,sub[SZ],cen[SZ],lev[SZ]; // subtree size, centroid anc
  int dist[32-__builtin_clz(SZ)][SZ]; // dists to all ancs
  vi stor[SZ], STOR[SZ];
  void dfs(int x, int p) { sub[x] = 1;
    each(y,adj[x]) if (!done[y] && y != p)
      dfs(y,x), sub[x] += sub[y];
  }
  int centroid(int x) {
    dfs(x,-1);
    for (int sz = sub[x];;) {
      pi mx = {0,0};
      each(y,adj[x]) if (!done[y] && sub[y] < sub[x])
        ckmax(mx,{sub[y],y});
      if (mx.f*2 <= sz) return x;
      x = mx.s;
    }
  }
  void genDist(int x, int p, int lev) {
    dist[lev][x] = dist[lev][p]+1;
    each(y,adj[x]) if (!done[y] && y != p) genDist(y,x,lev); }
  void gen(int CEN, int _x) { // CEN = centroid above x
    int x = centroid(_x); done[x] = 1; cen[x] = CEN;
    sub[x] = sub[_x]; lev[x] = (CEN == -1 ? 0 : lev[CEN]+1);
    dist[lev[x]][x] = 0;
    stor[x].rsz(sub[x]),STOR[x].rsz(sub[x]+1);
    each(y,adj[x]) if (!done[y]) genDist(y,x,lev[x]);
    each(y,adj[x]) if (!done[y]) gen(x,y);
  }
  void init(int _N) { N = _N; FOR(i,1,N+1) done[i] = 0;
    gen(-1,1); } // start at vert 1
  void upd(int x, int y) {
    int cur = x, pre = -1;
    R0F(i,lev[x]+1) {
      ad(stor[cur],y-dist[i][x]);
      if (pre != -1) ad(STOR[pre],y-dist[i][x]);
      if (i > 0) pre = cur, cur = cen[cur];
    }
  } // call propAll() after all updates
  void propAll() { FOR(i,1,N+1) prop(stor[i]), prop(STOR[i]); }
  int query(int x) { // get value at vertex x
    int cur = x, pre = -1, ans = 0;
    R0F(i,lev[x]+1) { // if pre != -1, subtract those from
      ans += stor[cur][dist[i][x]]; // same subtree
      if (pre != -1) ans -= STOR[pre][dist[i][x]];
      if (i > 0) pre = cur, cur = cen[cur];
    }
    return ans;
  }
};
```

7.2.1 SqrtDecompton

HLD generally suffices. If not, here are some common strategies:

- Rebuild the tree after every $\sqrt{N}$ queries.
- Consider vertices with $>$ or $<$ $\sqrt{N}$ degree separately.
- For subtree updates, note that there are $O(\sqrt{N})$ distinct sizes among child subtrees of any node.

Block Tree: Use a DFS to split edges into contiguous groups of size $\sqrt{N}$ to $2\sqrt{N}$.

Mo's Algorithm for Tree Paths: Maintain an array of vertices where each one appears twice, once when a DFS enters the vertex (`st`) and one when the DFS exists (`en`). For a tree path $u \leftrightarrow v$ such that `st[u]<st[v]`,

- If u is an ancestor of v, query `[st[u],st[v]]`.
- Otherwise, query `[en[u],st[v]]` and consider $LCA(u, v)$ separately.

Solutions with worse complexities can be faster if you optimize the operations that are performed most frequently. Use arrays instead of vectors whenever possible. Iterating over an array in order is faster than iterating through the same array in some other order (ex. one given by a random permutation) or DFSing on a tree of the same size. Also, the difference between $\sqrt{N}$ and the optimal block (or buffer) size can be quite large. Try up to 5x smaller or larger (at least).

7.3 DFS Algorithms

EulerPath.h

Description: Eulerian path starting at `src` if it exists, visits all edges exactly once. Works for both directed and undirected. Returns vector of {vertex,label of edge to vertex}. Second element of first pair is always -1.
Time: $\mathcal{O}(N + M)$

11eb8b, 34 lines

```
template<bool directed> struct Euler {
  int N; V<vector<pair<int, int>>> adj;
  V<vector<pair<int, int>>::iterator> its; vector<bool> used;
  Euler(int _N) {
    N = _N;
    adj.resize(N);
  }
  void ae(int u, int v) {
    int M = size(used); used.push_back(0);
    adj[u].push_back({v, M});
    if (!directed) adj[v].push_back({u, M});
  }
  vector<pair<int, int>> gen(int src = 0) {
    its.resize(N);
    for(int i = 0, i < N; i++) its[i] = begin(adj[i]);
    vector<pair<int, int>> ans, s{{src, -1}}; // {{vert,prev
      ↪vert},edge label}
    int lst = -1; // ans generated in reverse order
    while (size(s)) {
      int x = s.back().first;
      auto& it = its[x], en = end(adj[x]);
      while (it != en && used[it->s]) ++it;
      if (it == en) { // no more edges out of vertex
        if (lst != -1 && lst != x) return {};
        // not a path, no tour exists
        ans.push_back(s.back());
        s.pop_back();
        if (sz(s)) lst = ns.back().first;
      }
      else s.push_back(*it), used[it->s] = 1;
    } // must use all edges
    if (size(ans) != sz(used)+1) return {};
    ranges::reverse(ans); return ans;
  }
};
```

SCCT.h

Description: Tarjan's, DFS once to generate strongly connected components in topological order. a, b in same component if both $a \to b$ and $b \to a$ exist. Uses less memory than Kosaraju b/c doesn't store reverse edges.
Time: $\mathcal{O}(N + M)$

7b9c1d, 32 lines

```
struct SCC {
  int N, ti = 0;
  vector<vector<int>> adj;
  vector<int> disc, comp, stk, comps;
  SCC(int _N) {
    N = _N;
    adj.resize(N);
    disc.resize(N), comp.resize(N, -1);
  }
  void ae(int u, int v) { adj[u].push_back(v); }
  int dfs(int u) {
    int low = disc[u] = ++ti;
    stk.push_back(u);
    for (auto& v: adj[u]) if (comp[v] == -1) {
      low = !disc[v] ? min(low, dfs(v)) : min(low, disc[v]);
    }
    if (low == disc[u]) {
      comps.push_back(u);
      for (int v = -1; v != u;) {
        comp[v = stk.back()] = u;
        stk.pop_back();
      }
    }
    return low;
  }
  void gen() {
    for (int i = 0; i < N; i++) {
      if (!disc[i]) dfs(i);
    }
    ranges::reverse(comps);
  }
};
```

TECC.h

Description: Computes 2-edge-connected components (TECC) of an undirected graph using Tarjan's low-link algorithm. Two vertices belong to the same component iff there is no bridge on any path between them.

cbd4f6, 55 lines

```cpp
struct TECC {
  int N, ti = 0;
  vector<int> disc, low, comp;
  vector<vector<int>> adj, comps;
  TECC(int _N) {
    N = _N;
    disc.resize(N), low.resize(N);
    adj.resize(N), comp.resize(N, -1);
  }
  void ae(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  bool isBridge(int u, int v) {
    if (disc[u] > disc[v]) swap(u, v);
    return low[v] > disc[u];
  }
  void dfs(int u, int p) {
    bool pSkip = false;
    disc[u] = low[u] = ++ti;
    for (auto& v: adj[u]) {
      if (v == p && !pSkip) {
        pSkip = true;
        continue;
      }
      if (!disc[v]) {
        dfs(v, u);
        low[u] = min(low[u], low[v]);
        // if (isBridge(u, v)) {} // it's a bridge
      }
      else low[u] = min(low[u], disc[v]);
    }
  }
  void getComps(int u) {
    comps[comp[u]].push_back(u);
    for (auto& v: adj[u]) {
      if (~comp[v] || isBridge(u, v)) continue;
      comp[v] = comp[u];
      getComps(v);
    }
  }
  void gen() {
    for (int u = 0; u < N; u++) {
      if (!disc[u]) dfs(u, -1);
    }
    int id = 0;
    for (int u = 0; u < N; u++) {
      if (comp[u] == -1) {
        comp[u] = id++;
        comps.emplace_back();
        getComps(u);
      }
    }
  }
};
```

TwoSAT.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a\|\|b)\&\&(!a\|\|c)\&\&(d\|\|!b)\&\&\ldots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions ($\sim$x).

Usage:
```
TwoSat G(n);
ts.either(0, ~3); // Var 0 is true or var 3 is false
G.gen(N); // Returns true iff it is solvable
G.ans[0..N-1] holds the assigned values to the vars
```

"SCCK.h" 697509, 26 lines

```cpp
struct TwoSAT {
  int N;
  vector<bool> ans;
  vector<pair<int, int>> edges;
  TwoSAT(int _N) {
    N = _N;
    ans.resize(N);
  }
  void either(int u, int v) {
    u = max(2*u, -1-2*u), v = max(2*v, -1-2*v);
    edges.push_back({u, v});
  }
  bool gen() {
    SCC S(2*N);
    for (auto& [u, v]: edges) {
      S.ae(u^1, v);
      S.ae(v^1, u);
    }
    S.gen();
    for (int i = 0; i < N; i++) {
      if (S.comp[2*i] == S.comp[2*i^1]) return false;
      ans[i] = S.comp[2*i] > S.comp[2*i^1];
    }
    return true;
  }
};
```

BCC.h

Description: Biconnected components of edges. Removing any vertex in BCC doesn't disconnect it. To get block-cut tree, create a bipartite graph with the original vertices on the left and a vertex for each BCC on the right. Draw edge $u \leftrightarrow v$ if u is contained within the BCC for v. Self-loops are not included in any BCC while BCCS of size 1 represent bridges.
Time: $\mathcal{O}(N+M)$

"stdc++.h" ab2730, 45 lines

```cpp
using namespace std;

struct BCC {
  vector<pair<int, int>> ed;
  vector<vector<pair<int, int>>> adj;
  vector<vector<int>> edgeSets, vertSets; // edges for each bcc
  int N, ti = 0;
  vector<int> disc, stk;
  BCC(int _N) {
    N = _N;
    disc.resize(N), adj.resize(N);
  }
  void ae(int u, int v) {
    adj[u].emplace_back(v, size(ed));
    adj[v].emplace_back(u, size(ed));
    ed.emplace_back(u, v);
  }
  int dfs(int u, int p = -1) { // return lowest disc
    int low = disc[u] = ++ti;
    for (auto e : adj[u]) if (e.s != p) {
      if (!disc[e.f]) {
        stk.pb(e.s); // disc[x] < LOW -> bridge
        int LOW = dfs(e.f,e.s); ckmin(low,LOW);
        if (disc[x] <= LOW) { // get edges in bcc
          edgeSets.eb(); vi& tmp = edgeSets.bk; // new bcc
          for (int y = -1; y != e.s; )
            tmp.pb(y = stk.bk), stk.pop_back();
        }
      } else if (disc[e.f] < disc[x]) // back-edge
        ckmin(low,disc[e.f]), stk.pb(e.s);
    }
    return low;
  }
  void gen() {
    F0R(i,N) if (!disc[i]) dfs(i);
    vb in(N);
    each(c,edgeSets) { // edges contained within each BCC
      vertSets.eb(); // so you can easily create block cut tree
      auto ad = [&](int x) {
        if (!in[x]) in[x] = 1, vertSets.bk.pb(x); };
      each(e,c) ad(ed[e].f), ad(ed[e].s);
      each(e,c) in[ed[e].f] = in[ed[e].s] = 0;
    }
  }
};
```

MaximalCliques.h

Description: Used only once. Finds all maximal cliques.
Time: $\mathcal{O}\left(3^{N/3}\right)$

f5cd93, 16 lines

```cpp
using B = bitset<128>; B adj[128];
int N;
// possibly in clique, not in clique, in clique
void cliques(B P = ~B(), B X={}, B R={}) {
  if (!P.any()) {
    if (!X.any()) // do smth with R
    return;
  }
  int q = (P|X)._Find_first();
  // clique must contain q or non-neighbor of q
  B cands = P&~adj[q];
  F0R(i,N) if (cands[i]) {
    R[i] = 1; cliques(P&adj[i],X&adj[i],R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

7.4 DSU

DSU.h

Description: Disjoint Set Union with path compression and union by size. Add edges and test connectivity. Use for Kruskal's or Boruvka's minimum spanning tree.
Time: $\mathcal{O}(\alpha(N))$

543f93, 12 lines

```cpp
struct DSU {
  vector<int> e;
  DSU(int N) { e = vector<int>(N, -1); }
  int get(int u) { return e[u] < 0 ? u : e[u] = get(e[u]); }
  bool sameSet(int u, int v) { return get(u) == get(v); }
  int size(int u) { return -e[get(u)]; }
  bool unite(int u, int v) { // union by size
    u = get(u), v = get(v); if (u == v) return false;
    if (e[u] > e[v]) swap(u, v);
    e[u] += e[v]; e[v] = u; return true;
  }
}
```

Kruskal.h

Description: Computes the weight of a Minimum Spanning Tree (MST) using Kruskal's algorithm. Requires DSU.
Time: $\mathcal{O}(M \log M)$, where M = number of edges

"DSU.h" d32d13, 6 lines

```cpp
template <class T> T Kruskal(int N, vector<array<T, 3>> eg) {
  ranges::sort(eg);
  DSU D(N); T ans = 0;
  for (auto &[w, u, v] : eg) if (D.unite(u, v)) ans += w;
```

```
  return ans;
}
```

DSUrb.h

Description: Disjoint Set Union with Rollback

d21534, 25 lines

```
struct DSUrb {
  vector<int> e;
  vector<array<int,4>> mod;
  DSUrb(int n) { e = vector<int>(n,-1); }
  int get(int x) { return e[x] < 0 ? x : get(e[x]); }
  bool sameSet(int a, int b) { return get(a) == get(b); }
  int size(int x) { return -e[get(x)]; }
  bool unite(int x, int y) { // union-by-rank
    x = get(x), y = get(y);
    if (x == y) {
    mod.push_back({-1,-1,-1,-1});
    return 0;
    }
    if (e[x] > e[y]) swap(x,y);
    mod.push_back({x,y,e[x],e[y]});
    e[x] += e[y]; e[y] = x; return 1;
  }
  void rollback() {
    auto a = mod.back(); mod.pop_back();
    if (a[0] != -1) {
      e[a[0]] = a[2];
      e[a[1]] = a[3];
    }
  }
};
```

7.5 Flows

Konig's Theorem: In a bipartite graph, max matching = min vertex cover.

Dilworth's Theorem: For any partially ordered set, the sizes of the max antichain and of the min chain decomposition are equal. Equivalent to Konig's theorem on the bipartite graph (U, V, E) where $U = V = S$ and (u, v) is an edge when $u < v$. Those vertices outside the min vertex cover in both U and V form a max antichain.

Dinic.h

Description: Fast flow. After computing flow, edges $\{u, v\}$ such that $lev[u] \neq -1$, $lev[v] = -1$ are part of min cut. Use `reset` and `rcap` for Gomory-Hu.

Time: $\mathcal{O}\left(N^2M\right)$ flow, $\mathcal{O}\left(M\sqrt{N}\right)$ bipartite matching

b7b370, 38 lines

```
struct Dinic {
  using F = ll; // flow type
  struct Edge { int to; F flo, cap; };
  int N; V<Edge> eds; V<vi> adj;
  void init(int _N) { N = _N; adj.rsz(N), cur.rsz(N); }
  void ae(int u, int v, F cap, F rcap = 0) { assert(min(cap,
    ↪rcap) >= 0);
    adj[u].pb(sz(eds)); eds.pb({v,0,cap});
    adj[v].pb(sz(eds)); eds.pb({u,0,rcap});
  }
  vi lev; V<vi::iterator> cur;
  bool bfs(int s, int t) { // level = shortest distance from
    ↪source
    lev = vi(N,-1); F0R(i,N) cur[i] = begin(adj[i]);
    queue<int> q({s}); lev[s] = 0;
    while (sz(q)) { int u = q.ft; q.pop();
      each(e,adj[u]) { const Edge& E = eds[e];
        int v = E.to; if (lev[v] < 0 && E.flo < E.cap)
          q.push(v), lev[v] = lev[u]+1;
      }
    }
    return lev[t] >= 0;
  }
  F dfs(int v, int t, F flo) {
    if (v == t) return flo;
    for (; cur[v] != end(adj[v]); cur[v]++) {
      Edge& E = eds[*cur[v]];
      if (lev[E.to]!=lev[v]+1||E.flo==E.cap) continue;
      F df = dfs(E.to,t,min(flo,E.cap-E.flo));
      if (df) { E.flo += df; eds[*cur[v]^1].flo -= df;
        return df; } // saturated >=1 one edge
    }
    return 0;
  }
  F maxFlow(int s, int t) {
    F tot = 0; while (bfs(s,t)) while (F df =
      dfs(s,t,numeric_limits<F>::max())) tot += df;
    return tot;
  }
};
```

GomoryHu.h

Description: Returns edges of Gomory-Hu tree (second element is weight). Max flow between pair of vertices of undirected graph is given by min edge weight along tree path. Uses the fact that for any i, j, k, $\lambda_{ik} \geq \min(\lambda_{ij}, \lambda_{jk})$, where λ_{ij} denotes the flow between i and j.

Time: $N - 1$ calls to Dinic

"Dinic.h" 0d712e, 16 lines

```
template<class F> V<pair<pi,F>> gomoryHu(int N,
    const V<pair<pi,F>>& ed) {
  vi par(N); Dinic<F> D; D.init(N);
  vpi ed_locs; each(t,ed)ed_locs.pb(D.ae(t.f.f,t.f.s,t.s,t.s));
  V<pair<pi,F>> ans;
  FOR(i,1,N) {
    each(p,ed_locs) { // reset capacities
      auto& e = D.adj.at(p.f).at(p.s);
      auto& e_rev = D.adj.at(e.to).at(e.rev);
      e.cap = e_rev.cap = (e.cap+e_rev.cap)/2;
    }
    ans.pb({{i,par[i]},D.maxFlow(i,par[i])});
    FOR(j,i+1,N) if (par[j] == par[i] && D.lev[j]) par[j] = i;
  }
  return ans;
}
```

MCMF.h

Description: Minimum-cost maximum flow, assumes no negative cycles. It is possible to choose negative edge costs such that the first run of Dijkstra is slow, but this hasn't been an issue in the past. Edge weights ≥ 0 for every subsequent run. To get flow through original edges, assign ID's during `ae`.

Time: Ignoring first run of Dijkstra, $\mathcal{O}(FM \log M)$ if caps are integers and F is max flow.

072c1b, 40 lines

```
struct MCMF {
  using F = ll; using C = ll; // flow type, cost type
  struct Edge { int to; F flo, cap; C cost; };
  int N; V<C> p, dist; vi pre; V<Edge> eds; V<vi> adj;
  void init(int _N) { N = _N;
    p.rsz(N), dist.rsz(N), pre.rsz(N), adj.rsz(N); }
  void ae(int u, int v, F cap, C cost) { assert(cap >= 0);
    adj[u].pb(sz(eds)); eds.pb({v,0,cap,cost});
    adj[v].pb(sz(eds)); eds.pb({u,0,0,-cost});
  } // use asserts, don't try smth dumb
  bool path(int s, int t) { // find lowest cost path to send
    ↪flow through
    const C inf = numeric_limits<C>::max(); F0R(i,N) dist[i] =
      ↪inf;
    using T = pair<C,int>; priority_queue<T,vector<T>,greater<T
      ↪>> todo;
    todo.push({dist[s] = 0,s});
    while (sz(todo)) { // Dijkstra
      T x = todo.top(); todo.pop(); if (x.f > dist[x.s])
        ↪continue;
      each(e,adj[x.s]) { const Edge& E = eds[e]; // all weights
        ↪ should be non-negative
        if (E.flo < E.cap && ckmin(dist[E.to],x.f+E.cost+p[x.s
          ↪]-p[E.to]))
          pre[E.to] = e, todo.push({dist[E.to],E.to});
      }
    } // if costs are doubles, add some EPS so you
    // don't traverse ~0-weight cycle repeatedly
    return dist[t] != inf; // return flow
  }
  pair<F,C> calc(int s, int t) { assert(s != t);
    F0R(_,N) F0R(e,sz(eds)) { const Edge& E = eds[e]; //
      ↪Bellman-Ford
      if (E.cap) ckmin(p[E.to],p[eds[e^1].to]+E.cost); }
    F totFlow = 0; C totCost = 0;
    while (path(s,t)) { // p -> potentials for Dijkstra
      F0R(i,N) p[i] += dist[i]; // don't matter for unreachable
        ↪ nodes
      F df = numeric_limits<F>::max();
      for (int x = t; x != s; x = eds[pre[x]^1].to) {
        const Edge& E = eds[pre[x]]; ckmin(df,E.cap-E.flo); }
      totFlow += df; totCost += (p[t]-p[s])*df;
      for (int x = t; x != s; x = eds[pre[x]^1].to)
        eds[pre[x]].flo += df, eds[pre[x]^1].flo -= df;
    } // get max flow you can send along path
    return {totFlow,totCost};
  }
};
```

7.6 Matching

Hungarian.h

Description: Given J jobs and W workers (J <= W), computes the minimum cost to assign each prefix of jobs to distinct workers.
@tparam T a type large enough to represent integers on the order of J * max(—C—) @param C a matrix of dimensions JxW such that C[j][w] = cost to assign j-th job to w-th worker (possibly negative)
@return a vector of length J, with the j-th entry equaling the minimum cost to assign the first (j+1) jobs to distinct workers

Time: $\mathcal{O}\left(J^2W\right)$

f382f7, 36 lines

```
template <class T> vector<T> hungarian(const vector<vector<T>>
    ↪&C) {
  const int J = (int)size(C), W = (int)size(C[0]);
  assert(J <= W);
  vector<int> job(W + 1, -1);
  vector<T> ys(J), yt(W + 1);
  vector<T> answers;
  const T inf = numeric_limits<T>::max();
  for (int j_cur = 0; j_cur < J; ++j_cur) {
    int w_cur = W;
    job[w_cur] = j_cur;
    vector<T> min_to(W + 1, inf);
    vector<int> prv(W + 1, -1);
    vector<bool> in_Z(W + 1);
    while (job[w_cur] != -1) {
      in_Z[w_cur] = true;
      const int j = job[w_cur];
      T delta = inf;
      int w_next;
      for (int w = 0; w < W; ++w) {
        if (!in_Z[w]) {
```

```
          if (ckmin(min_to[w], C[j][w] - ys[j] - yt[w]))
            prv[w] = w_cur;
          if (ckmin(delta, min_to[w])) w_next = w;
        }
      }
      for (int w = 0; w <= W; ++w) {
        if (in_Z[w]) ys[job[w]] += delta, yt[w] -= delta;
        else min_to[w] -= delta;
      }
      w_cur = w_next;
    }
    for (int w; w_cur != -1; w_cur = w) job[w_cur] = job[w =
      ↪prv[w_cur]];
    answers.push_back(-yt[W]);
  }
  return answers;
}
```

GeneralMatchBlossom.h

Description: Variant on Gabow's Impl of Edmond's Blossom Algorithm. General unweighted max matching with 1-based indexing. If `white[v] = 0` after `solve()` returns, `v` is part of every max matching.
Time: $\mathcal{O}(NM)$, faster in practice

fd5cc7, 50 lines

```
struct MaxMatching {
  int N; V<vi> adj;
  V<int> mate, first; vb white; vpi label;
  void init(int _N) { N = _N; adj = V<vi>(N+1);
    mate = first = vi(N+1); label = vpi(N+1); white = vb(N+1);
      ↪}
  void ae(int u, int v) { adj.at(u).pb(v), adj.at(v).pb(u); }
  int group(int x) { if (white[first[x]]) first[x] = group(
    ↪first[x]);
    return first[x]; }
  void match(int p, int b) {
    swap(b,mate[p]); if (mate[b] != p) return;
    if (!label[p].s) mate[b] = label[p].f, match(label[p].f,b);
      ↪ // vertex label
    else match(label[p].f,label[p].s), match(label[p].s,label[p
      ↪].f); // edge label
  }
  bool augment(int st) { assert(st);
    white[st] = 1; first[st] = 0; label[st] = {0,0};
    queue<int> q; q.push(st);
    while (!q.empty()) {
      int a = q.ft; q.pop(); // outer vertex
      each(b,adj[a]) { assert(b);
        if (white[b]) { // two outer vertices, form blossom
          int x = group(a), y = group(b), lca = 0;
          while (x||y) {
            if (y) swap(x,y);
            if (label[x] == pi{a,b}) { lca = x; break; }
            label[x] = {a,b}; x = group(label[mate[x]].first);
          }
          for (int v: {group(a),group(b)}) while (v != lca) {
            assert(!white[v]); // make everything along path
              ↪white
            q.push(v); white[v] = true; first[v] = lca;
            v = group(label[mate[v]].first);
          }
        } else if (!mate[b]) { // found augmenting path
          mate[b] = a; match(a,b); white = vb(N+1); // reset
          return true;
        } else if (!white[mate[b]]) {
          white[mate[b]] = true; first[mate[b]] = b;
          label[b] = {0,0}; label[mate[b]] = pi{a,0};
          q.push(mate[b]);
        }
      }
    }
    return false;
  }
  int solve() {
    int ans = 0;
    FOR(st,1,N+1) if (!mate[st]) ans += augment(st);
    FOR(st,1,N+1) if (!mate[st] && !white[st]) assert(!augment(
      ↪st));
    return ans;
  }
};
```

GeneralWeightedMatch.h

Description: General max weight max matching with 1-based indexing. Edge weights must be positive, combo of UnweightedMatch and Hungarian.
Time: $\mathcal{O}(N^3)$?

120873, 145 lines

```
template<int SZ> struct WeightedMatch {
  struct edge { int u,v,w; }; edge g[SZ*2][SZ*2];
  void ae(int u, int v, int w) { g[u][v].w = g[v][u].w = w; }
  int N,NX,lab[SZ*2],match[SZ*2],slack[SZ*2],st[SZ*2];
  int par[SZ*2],floFrom[SZ*2][SZ],S[SZ*2],aux[SZ*2];
  vi flo[SZ*2]; queue<int> q;
  void init(int _N) { N = _N; // init all edges
    FOR(u,1,N+1) FOR(v,1,N+1) g[u][v] = {u,v,0}; }
  int eDelta(edge e) { // >= 0 at all times
    return lab[e.u]+lab[e.v]-g[e.u][e.v].w*2; }
  void updSlack(int u, int x) { // smallest edge -> blossom x
    if (!slack[x] || eDelta(g[u][x]) < eDelta(g[slack[x]][x]))
      slack[x] = u; }
  void setSlack(int x) {
    slack[x] = 0; FOR(u,1,N+1) if (g[u][x].w > 0
     && st[u] != x && S[st[u]] == 0) updSlack(u,x); }
  void qPush(int x) {
    if (x <= N) q.push(x);
    else each(t,flo[x]) qPush(t); }
  void setSt(int x, int b) {
    st[x] = b; if (x > N) each(t,flo[x]) setSt(t,b); }
  int getPr(int b, int xr) { // get even position of xr
    int pr = find(all(flo[b]),xr)-begin(flo[b]);
    if (pr&1) { reverse(1+all(flo[b])); return sz(flo[b])-pr; }
    return pr; }
  void setMatch(int u, int v) { // rearrange flo[u], matches
    edge e = g[u][v]; match[u] = e.v; if (u <= N) return;
    int xr = floFrom[u][e.u], pr = getPr(u,xr);
    F0R(i,pr) setMatch(flo[u][i],flo[u][i^1]);
    setMatch(xr,v); rotate(begin(flo[u]),pr+all(flo[u])); }
  void augment(int u, int v) { // set matches including u->v
    while (1) { // and previous ones
      int xnv = st[match[u]]; setMatch(u,v);
      if (!xnv) return;
      setMatch(xnv,st[par[xnv]]);
      u = st[par[xnv]], v = xnv;
    }
  }
  int lca(int u, int v) { // same as in unweighted
    static int t = 0; // except maybe return 0
    for (++t;u||v;swap(u,v)) {
      if (!u) continue;
      if (aux[u] == t) return u;
      aux[u] = t; u = st[match[u]];
      if (u) u = st[par[u]];
    }
    return 0;
  }
  void addBlossom(int u, int anc, int v) {
    int b = N+1; while (b <= NX && st[b]) ++b;
    if (b > NX) ++NX; // new blossom
    lab[b] = S[b] = 0; match[b] = match[anc]; flo[b] = {anc};
    auto blossom = [&](int x) {
      for (int y; x != anc; x = st[par[y]])
        flo[b].pb(x), flo[b].pb(y = st[match[x]]), qPush(y);
    };
    blossom(u); reverse(1+all(flo[b])); blossom(v); setSt(b,b);
    // identify all nodes in current blossom
    FOR(x,1,NX+1) g[b][x].w = g[x][b].w = 0;
    FOR(x,1,N+1) floFrom[b][x] = 0;
    each(xs,flo[b]) { // find tightest constraints
      FOR(x,1,NX+1) if (g[b][x].w == 0 || eDelta(g[xs][x]) <
        eDelta(g[b][x])) g[b][x]=g[xs][x], g[x][b]=g[x][xs];
      FOR(x,1,N+1) if (floFrom[xs][x]) floFrom[b][x] = xs;
    } // floFrom to deconstruct blossom
    setSlack(b); // since didn't qPush everything
  }
  void expandBlossom(int b) {
    each(t,flo[b]) setSt(t,t); // undo setSt(b,b)
    int xr = floFrom[b][g[b][par[b]].u], pr = getPr(b,xr);
    for(int i = 0; i < pr; i += 2) {
      int xs = flo[b][i], xns = flo[b][i+1];
      par[xs] = g[xns][xs].u; S[xs] = 1; // no setSlack(xns)?
      S[xns] = slack[xs] = slack[xns] = 0; qPush(xns);
    }
    S[xr] = 1, par[xr] = par[b];
    FOR(i,pr+1,sz(flo[b])) { // matches don't change
      int xs = flo[b][i]; S[xs] = -1, setSlack(xs); }
    st[b] = 0; // blossom killed
  }
  bool onFoundEdge(edge e) {
    int u = st[e.u], v = st[e.v];
    if (S[v] == -1) { // v unvisited, matched with smth else
      par[v] = e.u, S[v] = 1; slack[v] = 0;
      int nu = st[match[v]]; S[nu] = slack[nu] = 0; qPush(nu);
    } else if (S[v] == 0) {
      int anc = lca(u,v); // if 0 then match found!
      if (!anc) return augment(u,v),augment(v,u),1;
      addBlossom(u,anc,v);
    }
    return 0;
  }
  bool matching() {
    q = queue<int>();
    FOR(x,1,NX+1) {
      S[x] = -1, slack[x] = 0; // all initially unvisited
      if (st[x] == x && !match[x]) par[x] = S[x] = 0, qPush(x);
    }
    if (!sz(q)) return 0;
    while (1) {
      while (sz(q)) { // unweighted matching with tight edges
        int u = q.ft; q.pop(); if (S[st[u]] == 1) continue;
        FOR(v,1,N+1) if (g[u][v].w > 0 && st[u] != st[v]) {
          if (eDelta(g[u][v]) == 0) { // condition is strict
            if (onFoundEdge(g[u][v])) return 1;
          } else updSlack(u,st[v]);
        }
      }
      int d = INT_MAX;
      FOR(b,N+1,NX+1) if (st[b] == b && S[b] == 1)
        ckmin(d,lab[b]/2); // decrease lab[b]
      FOR(x,1,NX+1) if (st[x] == x && slack[x]) {
        if (S[x] == -1) ckmin(d,eDelta(g[slack[x]][x]));
        else if (S[x] == 0) ckmin(d,eDelta(g[slack[x]][x])/2);
      } // edge weights shouldn't go below 0
      FOR(u,1,N+1) {
        if (S[st[u]] == 0) {
          if (lab[u] <= d) return 0; // why?
          lab[u] -= d;
        } else if (S[st[u]] == 1) lab[u] += d;
      } // lab has opposite meaning for verts and blossoms
```

```
      FOR(b,N+1,NX+1) if (st[b] == b && S[b] != -1)
        lab[b] += (S[b] == 0 ? 1 : -1)*d*2;
      q = queue<int>();
      FOR(x,1,NX+1) if (st[x]==x && slack[x] // new tight edge
        && st[slack[x]] != x && eDelta(g[slack[x]][x]) == 0)
          if (onFoundEdge(g[slack[x]][x])) return 1;
      FOR(b,N+1,NX+1) if (st[b]==b && S[b]==1 && lab[b]==0)
        expandBlossom(b); // odd dist blossom taken apart
    }
    return 0;
  }
  pair<ll,int> calc() {
    NX = N; st[0] = 0; FOR(i,1,2*N+1) aux[i] = 0;
    FOR(i,1,N+1) match[i] = 0, st[i] = i, flo[i].clear();
    int wMax = 0;
    FOR(u,1,N+1) FOR(v,1,N+1)
      floFrom[u][v] = (u == v ? u : 0), ckmax(wMax,g[u][v].w);
    FOR(u,1,N+1) lab[u] = wMax; // start high and decrease
    int num = 0; ll wei = 0; while (matching()) ++num;
    FOR(u,1,N+1) if (match[u] && match[u] < u)
      wei += g[u][match[u]].w; // edges in matching
    return {wei,num};
  }
};
```

MaxMatchFast.h

Description: Fast bipartite matching.

Time: $\mathcal{O}\left(M\sqrt{N}\right)$

ec6c96, 31 lines

```
vpi maxMatch(int L, int R, const vpi& edges) {
  V<vi> adj = V<vi>(L);
  vi nxt(L,-1), prv(R,-1), lev, ptr;
  F0R(i,sz(edges)) adj.at(edges[i].f).pb(edges[i].s);
  while (true) {
    lev = ptr = vi(L); int max_lev = 0;
    queue<int> q; F0R(i,L) if (nxt[i]==-1) lev[i]=1, q.push(i);
    while (sz(q)) {
      int x = q.ft; q.pop();
      for (int y: adj[x]) {
        int z = prv[y];
        if (z == -1) max_lev = lev[x];
        else if (!lev[z]) lev[z] = lev[x]+1, q.push(z);
      }
      if (max_lev) break;
    }
    if (!max_lev) break;
    F0R(i,L) if (lev[i] > max_lev) lev[i] = 0;
    auto dfs = [&](auto self, int x) -> bool {
      for (;ptr[x] < sz(adj[x]);++ptr[x]) {
        int y = adj[x][ptr[x]], z = prv[y];
        if (z == -1 || (lev[z] == lev[x]+1 && self(self,z)))
          return nxt[x]=y, prv[y]=x, ptr[x]=sz(adj[x]), 1;
      }
      return 0;
    };
    F0R(i,L) if (nxt[i] == -1) dfs(dfs,i);
  }
  vpi ans; F0R(i,L) if (nxt[i] != -1) ans.pb({i,nxt[i]});
  return ans;
}
```

Geometry (8)

8.1 Primitives

PointShort.h

Description: Use in place of complex<T>.

268ecf, 36 lines

```
using T = db; // or ll
const T EPS = 1e-9; // adjust as needed
using P = pair<T,T>; using vP = V<P>; using Line = pair<P,P>;
int sgn(T a) { return (a>EPS)-(a<-EPS); }
T sq(T a) { return a*a; }

T abs2(P p) { return sq(p.f)+sq(p.s); }
T abs(P p) { return sqrt(abs2(p)); }
T arg(P p) { return atan2(p.s,p.f); }
P conj(P p) { return P(p.f,-p.s); }
P perp(P p) { return P(-p.s,p.f); }
P dir(T ang) { return P(cos(ang),sin(ang)); }

P operator+(P l, P r) { return P(l.f+r.f,l.s+r.s); }
P operator-(P l, P r) { return P(l.f-r.f,l.s-r.s); }
P operator*(P l, T r) { return P(l.f*r,l.s*r); }
P operator/(P l, T r) { return P(l.f/r,l.s/r); }
P operator*(P l, P r) { // complex # multiplication
  return P(l.f*r.f-l.s*r.s,l.s*r.f+l.f*r.s); }
P operator/(P l, P r) { return l*conj(r)/abs2(r); }

P unit(const P& p) { return p/abs(p); }
T dot(const P& a, const P& b) { return a.f*b.f+a.s*b.s; }
T dot(const P& p, const P& a, const P& b) { return dot(a-p,b-p)
   ↪; }
T cross(const P& a, const P& b) { return a.f*b.s-a.s*b.f; }
T cross(const P& p, const P& a, const P& b) {
  return cross(a-p,b-p); }
P reflect(const P& p, const Line& l) {
  P a = l.f, d = l.s-l.f;
  return a+conj((p-a)/d)*d; }
P foot(const P& p, const Line& l) {
  return (p+reflect(p,l))/(T)2; }
bool onSeg(const P& p, const Line& l) {
  return sgn(cross(l.f,l.s,p)) == 0 && sgn(dot(p,l.f,l.s)) <= 0
     ↪; }
ostream& operator<<(ostream& os, P p) {
  return os << "(" << p.f << "," << p.s << ")"; }
```

Strings (9)

9.1 Light

KMP.h

Description: f[i] is length of the longest proper suffix of the i-th prefix of s that is a prefix of s

Time: $\mathcal{O}(N)$

993fdd, 20 lines

```
vector<int> KMP(string s) {
  int N = s.size();
  vector<int> f(N + 1); f[0] = -1;
  for (int i = 1; i <= N; i++) {
    for (f[i] = f[i-1]; f[i] != -1 && s[f[i]] != s[i-1];) {
      f[i] = f[f[i]];
    }
  ++f[i];
  }
  return f;
}
vector<int> getOc(string a, string b){ // find occurrences of a
   ↪ in b
  vector<int> f = KMP(a + "@" + b), res;
  for (int i = a.size(); i <= b.size(); i++) {
    if (f[i+a.size()+1] == a.size()) {
      res.push_back(i - a.size());
    }
  }
  return res;
}
```

Z.h

Description: f[i] is the max len such that s.substr(0,len) == s.substr(i,len)

Usage: getPrefix("abcab","uwetrabcerabcab"));

Time: $\mathcal{O}(N)$

f97fbd, 16 lines

```
vector<int> Zfunction(string s){
  int N = s.size(), L = 1, R = 0; s += '#';
  vector<int> z(N); z[0] = N;
  for (int i = 1; i < N; i++) {
    if (i <= R) z[i] = min(R-i+1, z[i-L]);
    while (s[i+z[i]] == s[z[i]]) ++z[i];
    if (i+z[i]-1 > R) L = i, R = i+z[i]-1;
  }
  return z;
}
vector<int> getPrefix(string a, string b) { // find prefixes of
   ↪ a in b
  vector<int> t = Zfunction(a+b);
  t = vector<int>(size(a)+begin(t), end(t));
  for (auto& x: t) x = min(x, (int)size(a));
  return t;
}
```

Manacher.h

Description: length of largest palindrome centered at each character of string

Usage: ps(manacher("abacaba"))

Time: $\mathcal{O}(N)$ and between every consecutive pair

dccad2, 17 lines

```
vector<int> Manacher(string _S) {
  string S = "@";
  for (auto c: _S) S += c, S += "#";
  S.back() = '&';

  vector<int> ans(size(S)-1);
  for (int i = 1, lo = 0, hi = 0; i < size(S)-1; i++) {
    if (i != 1) ans[i] = min(hi-i, ans[hi-i+lo]);
    while (S[i-ans[i]-1] == S[i+ans[i]+1]) ++ans[i];
    if (i+ans[i] > hi) lo = i-ans[i], hi = i+ans[i];
  }
  ans.erase(begin(ans));
  for (int i = 0; i < size(ans); i++) {
    if (i%2 == ans[i]%2) ++ans[i];
  }
  return ans;
}
```

MinRotation.h

Description: minimum cyclic shift

Time: $\mathcal{O}(N)$

99d334, 12 lines

```
int minRotation(string s) {
  int a = 0, N = size(s); s += s;
  for (int b = 0; b < N; b++) {
    for (int i = 0; i < N; i++) {
      // a is current best rotation found up to b-1
      if (a+i == b || s[a+i] < s[b+i]) { b += max(0, i-1);
         ↪break; }
      // b to b+i-1 can't be better than a to a+i-1
```

```
      if (s[a+i] > s[b+i]) { a = b; break; } // new best found
    }
  }
  return a;
}
```

LyndonFactor.h

Description: A string is "simple" if it is strictly smaller than any of its own nontrivial suffixes. The Lyndon factorization of the string s is a factorization $s = w_1 w_2 \dots w_k$ where all strings w_i are simple and $w_1 \geq w_2 \geq \dots \geq w_k$. Min rotation gets min index i such that cyclic shift of s starting at i is minimum.
Time: $\mathcal{O}(N)$

af38ba, 19 lines

```
vs duval(str s) {
  int N = sz(s); vs factors;
  for (int i = 0; i < N; ) {
    int j = i+1, k = i;
    for (; j < N && s[k] <= s[j]; ++j) {
      if (s[k] < s[j]) k = i;
      else ++k;
    }
    for (; i <= k; i += j-k) factors.pb(s.substr(i,j-k));
  }
  return factors;
}
int minRotation(str s) {
  int N = sz(s); s += s;
  vs d = duval(s); int ind = 0, ans = 0;
  while (ans+sz(d[ind]) < N) ans += sz(d[ind++]);
  while (ind && d[ind] == d[ind-1]) ans -= sz(d[ind--]);
  return ans;
}
```

HashRange.h

Description: Polynomial hash for substrings with two bases.

7f6156, 50 lines

```
using H = array<int, 2>;
H makeH(char c) { return {c, c}; }
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    ↪count());
uniform_int_distribution<int> UID(0.1*MOD, 0.9*MOD);
const H base = {UID(rng), UID(rng)};
H operator+(H l, H r) {
  for (int i = 0; i < 2; i++) {
    if ((l[i] += r[i]) >= MOD) l[i] -= MOD;
  }
  return l;
}
H operator-(H l, H r) {
  for (int i = 0; i < 2; i++) {
    if ((l[i] -= r[i]) < 0) l[i] += MOD;
  }
  return l;
}
H operator*(H l, H r) {
  for (int i = 0; i < 2; i++) {
    l[i] = (1LL*l[i]*r[i]) % MOD;
  }
  return l;
}
vector<H> pows{{1, 1}};
struct HashRange {
  string S; vector<H> sum{{}};
  void add(char c) {
  S += c;
  sum.push_back(base*sum.back()+makeH(c));
  }
  void add(string s) {
    for (auto &c : s) add(c);
  }
  void extend(int len) {
    while ((int)pows.size() <= len) {
      pows.push_back(base * pows.back());
    }
  }
  H hash(int l, int r) {
    int len = r+1-l; extend(len);
    return sum[r+1] - pows[len] * sum[l];
  }
  uint64_t hash64(int l, int r) {
    H p = hash(l, r);
    return (uint64_t) p[0]<<32|p[1];
  }
  // int lcp(HashRange& b) { return first_true([&](int x) {
  //  return sum[x] != b.sum[x]; },0,min(sz(S),sz(b.S)))-1;
  // }
};
```

ReverseBW.h

Description: Used only once. Burrows-Wheeler Transform appends # to a string, sorts the rotations of the string in increasing order, and constructs a new string that contains the last character of each rotation. This function reverses the transform.
Time: $\mathcal{O}(N \log N)$

e400d8, 7 lines

```
str reverseBW(str t) {
  vi nex(sz(t)); iota(all(nex),0);
  stable_sort(all(nex),[&t](int a,int b){return t[a]<t[b];});
  str ret; for (int i = nex[0]; i; )
    ret += t[i = nex[i]];
  return ret;
}
```

AhoCorasickFixed.h

Description: Aho-Corasick for fixed alphabet. For each prefix, stores link to max length suffix which is also a prefix.
Time: $\mathcal{O}(N \sum)$

96dfcc, 27 lines

```
template<size_t ASZ> struct ACfixed {
  struct Node { AR<int, ASZ> to; int link; };
  V<Node> d{{}};
  int add(str s) { // add word
    int v = 0;
    each(C,s) {
      int c = C-'a';
      if (!d[v].to[c]) d[v].to[c] = sz(d), d.eb();
      v = d[v].to[c];
    }
    return v;
  }
  void init() { // generate links
    d[0].link = -1;
    queue<int> q; q.push(0);
    while (sz(q)) {
      int v = q.ft; q.pop();
      F0R(c,ASZ) {
        int u = d[v].to[c]; if (!u) continue;
        d[u].link = d[v].link == -1 ? 0 : d[d[v].link].to[c];
        q.push(u);
      }
      if (v) F0R(c,ASZ) if (!d[v].to[c])
        d[v].to[c] = d[d[v].link].to[c];
    }
  }
};
```

SuffixArray.h

Description: Sort suffixes. First element of `sa` is `sz(S)`, `isa` is the inverse of `sa`, and `lcp` stores the longest common prefix between every two consecutive elements of `sa`.
Time: $\mathcal{O}(N \log N)$

"RMQ.h" 27a566, 30 lines

```
struct SuffixArray {
  str S; int N; vi sa, isa, lcp;
  void init(str _S) { N = sz(S = _S)+1; genSa(); genLcp(); }
  void genSa() { // sa has size sz(S)+1, starts with sz(S)
    sa = isa = vi(N); sa[0] = N-1; iota(1+all(sa),0);
    sort(1+all(sa),[&](int a, int b) { return S[a] < S[b]; });
    FOR(i,1,N) { int a = sa[i-1], b = sa[i];
      isa[b] = i > 1 && S[a] == S[b] ? isa[a] : i; }
    for (int len = 1; len < N; len *= 2) { // currently sorted
      // by first len chars
      vi s(sa), is(isa), pos(N); iota(all(pos),0);
      each(t,s) {int T=t-len;if (T>=0) sa[pos[isa[T]]++] = T;}
      FOR(i,1,N) { int a = sa[i-1], b = sa[i];
        isa[b] = is[a]==is[b]&&is[a+len]==is[b+len]?isa[a]:i; }
    }
  }
  void genLcp() { // Kasai's Algo
    lcp = vi(N-1); int h = 0;
    F0R(b,N-1) { int a = sa[isa[b]-1];
      while (a+h < sz(S) && S[a+h] == S[b+h]) ++h;
      lcp[isa[b]-1] = h; if (h) h--; }
    R.init(lcp);
  }
  RMQ<int> R;
  int getLCP(int a, int b) { // lcp of suffixes starting at a,b
    if (a == b) return sz(S)-a;
    int l = isa[a], r = isa[b]; if (l > r) swap(l,r);
    return R.query(l,r-1);
  }
};
```

SuffixArrayLinear.h

Description: Linear-time suffix array.
Usage: `sa_is(s, 26) // all entries must be in [0, 26)`
Time: $\mathcal{O}(N)$, ~100ms for $N = 5 \cdot 10^5$

ed0bb4, 46 lines

```
vi sa_is(const vi& s, int upper) {
  int n = sz(s); if (!n) return {};
  vi sa(n); vb ls(n);
  R0F(i,n-1) ls[i] = s[i] == s[i+1] ? ls[i+1] : s[i] < s[i+1];
  vi sum_l(upper), sum_s(upper);
  F0R(i,n) (ls[i] ? sum_l[s[i]+1] : sum_s[s[i]])++;
  F0R(i,upper) {
    if (i) sum_l[i] += sum_s[i-1];
    sum_s[i] += sum_l[i];
  }
  auto induce = [&](const vi& lms) {
    fill(all(sa),-1);
    vi buf = sum_s;
    for (int d: lms) if (d != n) sa[buf[s[d]]++] = d;
    buf = sum_l; sa[buf[s[n-1]]++] = n-1;
    F0R(i,n) {
      int v = sa[i]-1;
      if (v >= 0 && !ls[v]) sa[buf[s[v]]++] = v;
    }
    buf = sum_l;
    R0F(i,n) {
      int v = sa[i]-1;
      if (v >= 0 && ls[v]) sa[--buf[s[v]+1]] = v;
    }
  };
  vi lms_map(n+1,-1), lms; int m = 0;
  FOR(i,1,n) if (!ls[i-1] && ls[i]) lms_map[i]=m++, lms.pb(i);
```

```
  induce(lms); // sorts LMS prefixes
  vi sorted_lms;each(v,sa)if (lms_map[v]!=-1)sorted_lms.pb(v);
  vi rec_s(m); int rec_upper = 0; // smaller subproblem
  FOR(i,1,m) { // compare two lms substrings in sorted order
    int l = sorted_lms[i-1], r = sorted_lms[i];
    int end_l = lms_map[l]+1 < m ? lms[lms_map[l]+1] : n;
    int end_r = lms_map[r]+1 < m ? lms[lms_map[r]+1] : n;
    bool same = 0; // whether lms substrings are same
    if (end_l-l == end_r-r) {
      for (;l < end_l && s[l] == s[r]; ++l,++r);
      if (l != n && s[l] == s[r]) same = 1;
    }
    rec_s[lms_map[sorted_lms[i]]] = (rec_upper += !same);
  }
  vi rec_sa = sa_is(rec_s,rec_upper+1);
  F0R(i,m) sorted_lms[i] = lms[rec_sa[i]];
  induce(sorted_lms); // sorts LMS suffixes
  return sa;
}
```

TandemRepeats.h

Description: Find all (i,p) such that `s.substr(i,p) == s.substr(i+p,p)`. No two intervals with the same period intersect or touch.

Usage: `solve("aaababababa") // {{0, 1, 1}, {2, 5, 2}}`

Time: $\mathcal{O}(N \log N)$

"SuffixArray.h" 661326, 13 lines

```
V<AR<int,3>> solve(str s) {
  int N = sz(s); SuffixArray A,B;
  A.init(s); reverse(all(s)); B.init(s);
  V<AR<int,3>> runs;
  for (int p = 1; 2*p <= N; ++p) { // do in O(N/p) for period p
    for (int i = 0, lst = -1; i+p <= N; i += p) {
      int l = i-B.getLCP(N-i-p,N-i), r = i-p+A.getLCP(i,i+p);
      if (l > r || l == lst) continue;
      runs.pb({lst = l,r,p}); // for each i in [l,r],
    } // s.substr(i,p) == s.substr(i+p,p)
  }
  return runs;
}
```

9.2 Heavy

PalTree.h

Description: Used infrequently. Palindromic tree computes number of occurrences of each palindrome within string. `ans[i][0]` stores min even x such that the prefix $s[1..i]$ can be split into exactly x palindromes, `ans[i][1]` does the same for odd x.

Time: $\mathcal{O}(N\sum)$ for addChar, $\mathcal{O}(N \log N)$ for updAns

8a7d31, 41 lines

```
struct PalTree {
  static const int ASZ = 26;
  struct node {
    AR<int,ASZ> to = AR<int,ASZ>();
    int len, link, oc = 0; // # occurrences of pal
    int slink = 0, diff = 0;
    AR<int,2> seriesAns;
    node(int _len, int _link) : len(_len), link(_link) {}
  };
  str s = "@"; V<AR<int,2>> ans = {{0,MOD}};
  V<node> d = {{0,1},{-1,0}}; // dummy pals of len 0,-1
  int last = 1;
  int getLink(int v) {
    while (s[sz(s)-d[v].len-2] != s.bk) v = d[v].link;
    return v;
  }
  void updAns() { // serial path has O(log n) vertices
    ans.pb({MOD,MOD});
    for (int v = last; d[v].len > 0; v = d[v].slink) {
      d[v].seriesAns=ans[sz(s)-1-d[d[v].slink].len-d[v].diff];
      if (d[v].diff == d[d[v].link].diff)
        F0R(i,2) ckmin(d[v].seriesAns[i],
              d[d[v].link].seriesAns[i]);
      // start of previous oc of link[v]=start of last oc of v
      F0R(i,2) ckmin(ans.bk[i],d[v].seriesAns[i^1]+1);
    }
  }
  void addChar(char C) {
    s += C; int c = C-'a'; last = getLink(last);
    if (!d[last].to[c]) {
      d.eb(d[last].len+2,d[getLink(d[last].link)].to[c]);
      d[last].to[c] = sz(d)-1;
      auto& z = d.bk; z.diff = z.len-d[z.link].len;
      z.slink = z.diff == d[z.link].diff
        ? d[z.link].slink : z.link;
    } // max suf with different dif
    last = d[last].to[c]; ++d[last].oc;
    updAns();
  }
  void numOc() { ROF(i,2,sz(d)) d[d[i].link].oc += d[i].oc; }
};
```

SuffixAutomaton.h

Description: Used infrequently. Constructs minimal deterministic finite automaton (DFA) that recognizes all suffixes of a string. `len` corresponds to the maximum length of a string in the equivalence class, `pos` corresponds to the first ending position of such a string, `lnk` corresponds to the longest suffix that is in a different class. Suffix links correspond to suffix tree of the reversed string!

Time: $\mathcal{O}(N \log \sum)$

a99c6d, 67 lines

```
struct SuffixAutomaton {
  int N = 1; vi lnk{-1}, len{0}, pos{-1}; // suffix link,
  // max length of state, last pos of first occurrence of state
  V<map<char,int>> nex{1}; V<bool> isClone{0};
  // transitions, cloned -> not terminal state
  V<vi> iLnk; // inverse links
  int add(int p, char c) { // ~p nonzero if p != -1
    auto getNex = [&]() {
      if (p == -1) return 0;
      int q = nex[p][c]; if (len[p]+1 == len[q]) return q;
      int clone = N++; lnk.pb(lnk[q]); lnk[q] = clone;
      len.pb(len[p]+1), nex.pb(nex[q]),
      pos.pb(pos[q]), isClone.pb(1);
      for (; ~p && nex[p][c] == q; p = lnk[p]) nex[p][c]=clone;
      return clone;
    };
    // if (nex[p].count(c)) return getNex();
    // ^ need if adding > 1 string
    int cur = N++; // make new state
    lnk.eb(), len.pb(len[p]+1), nex.eb(),
    pos.pb(pos[p]+1), isClone.pb(0);
    for (; ~p && !nex[p].count(c); p = lnk[p]) nex[p][c] = cur;
    int x = getNex(); lnk[cur] = x; return cur;
  }
  void init(str s) { int p = 0; each(x,s) p = add(p,x); }
  // inverse links
  void genIlnk() {iLnk.rsz(N);FOR(v,1,N)iLnk[lnk[v]].pb(v);}
  // APPLICATIONS
  void getAllOccur(vi& oc, int v) {
    if (!isClone[v]) oc.pb(pos[v]); // terminal position
    each(u,iLnk[v]) getAllOccur(oc,u); }
  vi allOccur(str s) { // get all occurrences of s in automaton
    int cur = 0;
    each(x,s) {
      if (!nex[cur].count(x)) return {};
      cur = nex[cur][x]; }
    // convert end pos -> start pos
    vi oc; getAllOccur(oc,cur); each(t,oc) t += 1-sz(s);
    sort(all(oc)); return oc;
  }
  vl distinct;
  ll getDistinct(int x) {
    // # distinct strings starting at state x
    if (distinct[x]) return distinct[x];
    distinct[x]=1;each(y,nex[x]) distinct[x]+=getDistinct(y.s);
    return distinct[x]; }
  ll numDistinct() { // # distinct substrings including empty
    distinct.rsz(N); return getDistinct(0); }
  ll numDistinct2() { // assert(numDistinct()==numDistinct2());
    ll ans = 1; FOR(i,1,N) ans += len[i]-len[lnk[i]];
    return ans; }
};

SuffixAutomaton S;
vi sa; str s;
void dfs(int x) {
  if (!S.isClone[x]) sa.pb(sz(s)-1-S.pos[x]);
  V<pair<char,int>> chr;
  each(t,S.iLnk[x]) chr.pb({s[S.pos[t]-S.len[x]],t});
  sort(all(chr)); each(t,chr) dfs(t.s);
}

int main() {
  re(s); reverse(all(s));
  S.init(s); S.genIlnk();
  dfs(0); ps(sa); // generating suffix array for s
}
```

SuffixTree.h

Description: Used infrequently. Ukkonen's algorithm for suffix tree. Longest non-unique suffix of `s` has length `len[p]+lef` after each call to `add` terminates. Each iteration of loop within `add` decreases this quantity by one.

Time: $\mathcal{O}(N \log \sum)$

39751c, 51 lines

```
struct SuffixTree {
  str s; int N = 0;
  vi pos, len, lnk; V<map<char,int>> to;
  int make(int POS, int LEN) { // lnk[x] is meaningful when
    // x!=0 and len[x] != MOD
    pos.pb(POS);len.pb(LEN);lnk.pb(-1);to.eb();return N++; }
  void add(int& p, int& lef, char c) { // longest
    // non-unique suffix is at node p with lef extra chars
    s += c; ++lef; int lst = 0;
    for (;lef;p?p=lnk[p]:lef--) { // if p != root then lnk[p]
      // must be defined
      while (lef>1 && lef>len[to[p][s[sz(s)-lef]]])
        p = to[p][s[sz(s)-lef]], lef -= len[p];
      // traverse edges of suffix tree while you can
      char e = s[sz(s)-lef]; int& q = to[p][e];
      // next edge of suffix tree
      if (!q) q = make(sz(s)-lef,MOD), lnk[lst] = p, lst = 0;
      // make new edge
      else {
        char t = s[pos[q]+lef-1];
        if (t == c) { lnk[lst] = p; return; } // suffix not
          ↪unique
        int u = make(pos[q],lef-1);
        // new node for current suffix-1, define its link
        to[u][c] = make(sz(s)-1,MOD); to[u][t] = q;
        // new, old nodes
        pos[q] += lef-1; if (len[q] != MOD) len[q] -= lef-1;
        q = u, lnk[lst] = u, lst = u;
      }
    }
  }
  void init(str _s) {
```

```
    make(-1,0); int p = 0, lef = 0;
    each(c,_s) add(p,lef,c);
    add(p,lef,'$'); s.pop_back(); // terminal char
  }
  int maxPre(str x) { // max prefix of x which is substring
    for (int p = 0, ind = 0;;) {
      if (ind == sz(x) || !to[p].count(x[ind])) return ind;
      p = to[p][x[ind]];
      F0R(i,len[p]) {
        if (ind == sz(x) || x[ind] != s[pos[p]+i]) return ind;
        ind ++;
      }
    }
  }
  vi sa; // generate suffix array
  void genSa(int x = 0, int Len = 0) {
    if (!sz(to[x])) sa.pb(pos[x]-Len); // found terminal node
    else each(t,to[x]) genSa(t.s,Len+len[x]);
  }
};
```

Various (10)

10.1 Searching

fstTrue.h

Description: Finds the first value x in [lo, hi] such that predicate f(x) is true.
Usage: `int ans = fstTrue()`
Time: $\mathcal{O}(\log(hi - lo))$

2654fa, 9 lines

```
template<typename T, typename U> T fstTrue(T lo, T hi, U f) {
  ++hi;
  assert(lo <= hi);
  while (lo < hi) {
    T mid = lo + (hi - lo) / 2;
    f(mid) ? hi = mid : lo = mid + 1;
  }
  return lo;
};
```

lstTrue.h

Description: Finds the last value x in [lo, hi] such that predicate f(x) is true.
Usage: `int ans = fstTrue()`
Time: $\mathcal{O}(\log(hi - lo))$

8aff85, 9 lines

```
template<typename T, typename U> T lstTrue(T lo, T hi, U f) {
  --lo;
  assert(lo <= hi);
  while (lo < hi) {
    T mid = lo + (hi - lo + 1) / 2;
    f(mid) ? lo = mid : hi = mid - 1;
  }
  return lo;
};
```

realTrue.h

Description: Binary search on continuous (floating-point) domain.
Usage: `int ans = realTrue()`
Time: $\mathcal{O}(\log(precision))$ — write 100 iterations and have a relax

691e1d, 7 lines

```
template<typename T, typename U>T realTrue(T lo, T hi, U f) {
  for (int i = 0; i < 100; ++i) {
    T mid = 0.5 * (hi + lo);
    f(mid) ? lo = mid : hi = mid;
  }
  return lo;
```

```
};
```

Ternary.h

Description: Ternary search to find the min of a unimodal function
Time: $\mathcal{O}(\log(N/eps))$

ef144f, 9 lines

```
template<typename U>
double Ternary(double lo, double hi, U f) {
  while (hi - lo > eps) {
    double l = lo + (hi - lo) / 3;
    double r = hi - (hi - lo) / 3;
    f(l) > f(r) ? lo = l : hi = r;
  }
  return lo;
}
```

10.2 Dynamic programming

When doing DP on intervals:
$a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j,

- one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$.
- This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$.
- Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

CircularLCS.h

Description: Used only twice. For strs A, B calculates longest common subsequence of A with all rotations of B
Time: $\mathcal{O}(|A| \cdot |B|)$

db21cf, 26 lines

```
int circular_lcs(str A, str B) {
  B += B;
  int max_lcs = 0;
  V<vb> dif_left(sz(A)+1,vb(sz(B)+1)), dif_up(sz(A)+1,vb(sz(B)
    ↪+1));
  auto recalc = [&](int x, int y) { assert(x && y);
    int res = (A.at(x-1) == B.at(y-1)) |
      dif_up[x][y-1] | dif_left[x-1][y];
    dif_left[x][y] = res-dif_up[x][y-1];
    dif_up[x][y]   = res-dif_left[x-1][y];
  };
  FOR(i,1,sz(A)+1) FOR(j,1,sz(B)+1) recalc(i,j);
  F0R(j,sz(B)/2) {
    // 1. zero out dp[.][j], update dif_left and dif_right
    if (j) for (int x = 1, y = j; x <= sz(A) && y <= sz(B); ) {
      int pre_up = dif_up[x][y];
      if (y == j) dif_up[x][y] = 0;
      else recalc(x,y);
      (pre_up == dif_up[x][y]) ? ++x : ++y;
    }
    // 2. calculate LCS(A[0:sz(A)),B[j:j+sz(B)/2))
    int cur_lcs = 0;
    FOR(x,1,sz(A)+1) cur_lcs += dif_up[x][j+sz(B)/2];
    ckmax(max_lcs,cur_lcs);
  }
  return max_lcs;
}
```

SMAWK.h

Description: Given negation of totally monotone matrix with entries of type D, find indices of row maxima (their indices increase for every submatrix). If tie, take lesser index. f returns matrix entry at (r,c) in $O(1)$. Use in place of divide & conquer to remove a log factor.
Time: $\mathcal{O}(R + C)$, can be reduced to $\mathcal{O}(C(1 + \log R/C))$ evaluations of f

0a9abb, 25 lines

```
template<class F, class D=ll> vi smawk (F f, vi x, vi y) {
  vi ans(sz(x),-1); // x = rows, y = cols
  #define upd() if (ans[i] == -1 || w > mx) ans[i] = c, mx = w
  if (min(sz(x),sz(y)) <= 8) {
    F0R(i,sz(x)) { int r = x[i]; D mx;
      each(c,y) { D w = f(r,c); upd(); } }
    return ans;
  }
  if (sz(x) < sz(y)) { // reduce subset of cols to consider
    vi Y; each(c,y) {
      for (;sz(Y);Y.pop_back()) { int X = x[sz(Y)-1];
        if (f(X,Y.bk) >= f(X,c)) break; }
      if (sz(Y) < sz(x)) Y.pb(c);
    } y = Y;
  } // recurse on half the rows
  vi X; for (int i = 1; i < sz(x); i += 2) X.pb(x[i]);
  vi ANS = smawk(f,X,y); F0R(i,sz(ANS)) ans[2*i+1] = ANS[i];
  for (int i = 0, k = 0; i < sz(x); i += 2){
    int to = i+1 < sz(ans) ? ans[i+1] : y.bk; D mx;
    for(int r = x[i];;++k) {
      int c = y[k]; D w = f(r,c); upd();
      if (c == to) break; }
  }
  return ans;
};
```

10.3 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` violations generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (`1`), 0-divs (`4`), infinities (`8`) and denormals (`16`).

10.4 Optimization tricks

10.4.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `F0R(b,k) F0R(i,1<<K) if (i&1<<b) D[i] += D[i^(1<<b)];` computes all sums of subsets.

10.4.2 STL Utilities

- `template<class T> using priority_queue_min = ↪priority_queue<T, vector<T>, greater<T>>;` Min-heap priority queue.

- `ranges::rotate(p, ranges::min_element(p));` Lexicographically smallest cyclic shift.
- `next_permutation(v.begin(), v.end());` Next permutation.
- `prev_permutation(v.begin(), v.end());` Previous permutation.
- `auto [mn, mx] = ranges::minmax(v);` Min and max in one pass.
- `ranges::nth_element(v, v.begin()+k);` k-th smallest in $O(n)$.
- `ranges::binary_search(v, x);` Check existence in sorted vector.
- `ranges::lower_bound(v, x);` First element $>= x$.
- `ranges::upper_bound(v, x);` First element $> x$.
- `int cnt = ranges::upper_bound(v, R)- ranges::lower_bound(v, L);` Count of elements in range $[L, R]$.
- `auto [l,r] = equal_range(v.begin(),v.end(),x);` Range of equal values.
- `v.erase(unique(v.begin(),v.end()),v.end());` Remove duplicates (after sort).
- `iota(v.begin(),v.end(),0);` Fill with increasing values.
- Prefix sum

```
inclusive_scan(v.begin(), v.end(), pre.begin()+1,
[](auto sum, auto x){ return sum + x; }, 0LL
);
```

10.4.3 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize for loops and optimizes floating points better (assumes associativity and turns off denormals).
- `#pragma GCC target ("avx,avx2")` can double performance of vectorized code, but causes crashes on old machines. Also consider older `#pragma GCC target ("sse4")`.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastIO.h
Description: Fast input and output for integers and strings. For doubles, read them as strings and convert them to double using `stod`.
Usage: `initO(); int a,b; ri(a,b); wi(b,'\n'); wi(a,'\n');`
Time: input is ~300ms faster for 10^6 long longs on CF.
cbf279, 39 lines

```
inline namespace FastIO {

const int BSZ = 1<<15; ////// INPUT
char ibuf[BSZ]; int ipos, ilen;
char nc() { // next char
  if (ipos == ilen) {
    ipos = 0; ilen = fread(ibuf,1,BSZ,stdin);
    if (!ilen) return EOF;
  }
  return ibuf[ipos++];
}
void rs(str& x) { // read str
  char ch; while (isspace(ch = nc()));
  do { x += ch; } while (!isspace(ch = nc()) && ch != EOF);
}
tcT> void ri(T& x) { // read int or ll
  char ch; int sgn = 1;
  while (!isdigit(ch = nc())) if (ch == '-') sgn *= -1;
  x = ch-'0'; while (isdigit(ch = nc())) x = x*10+(ch-'0');
  x *= sgn;
}
tcT, class... Ts> void ri(T& t, Ts&... ts) {
  ri(t); ri(ts...); } // read ints
////// OUTPUT (call initO() at start)
char obuf[BSZ], numBuf[100]; int opos;
void flushOut() { fwrite(obuf,1,opos,stdout); opos = 0; }
void wc(char c) { // write char
  if (opos == BSZ) flushOut();
  obuf[opos++] = c; }
void ws(str s) { each(c,s) wc(c); } // write str
tcT> void wi(T x, char after = '\0') {
  if (x < 0) wc('-'), x *= -1;
  int len = 0; for (;x>=10;x/=10) numBuf[len++] = '0'+(x%10);
  wc('0'+x); R0F(i,len) wc(numBuf[i]);
  if (after) wc(after);
}
void initO() { assert(atexit(flushOut) == 0); }

}
```

10.5 Other languages

Python3.py
Description: Python review.
40 lines

```
from math import *
import sys, random
def nextInt():
  return int(input())
def nextStrs():
  return input().split()
def nextInts():
  return list(map(int,nextStrs()))

n = nextInt()
v = [n]
def process(x):
  global v
  x = abs(x)
  V = []
  for t in v:
    g = gcd(t,x)
    if g != 1:
      V.append(g)
    if g != t:
      V.append(t//g)
  v = V
for i in range(50):
  x = random.randint(0,n-1)
  if gcd(x,n) != 1:
    process(x)
  else:
    sx = x*x%n # assert(gcd(sx,n) == 1)
    print(f"sqrt {sx}")
    sys.stdout.flush()
    X = nextInt()
    process(x+X)
    process(x-X)
print(f'! {len(v)}',end='')
for i in v:
  print(f' {i}',end='')
print()
sys.stdout.flush() # sys.exit(0) -> exit
# sys.setrecursionlimit(int(1e9)) -> stack size
# print(f'{ans:=.6f}') -> print ans to 6 decimal places
```

Decimal.py
Description: Arbitrary-precision decimals
5 lines

```
from decimal import *

getcontext().prec = 100 # how many digits of precision
print(Decimal(1) / Decimal(7)) # 0.142857142857...
print(Decimal(10) ** -100) # 1E-100
```

10.6 Miscellaneous

GridDirection.h
Description: Predefined direction for grid-based problems. Includes common movement sets for 2D and 3D grids such as 4-directional, 8-directional, diagonal-only, knight moves, 3D etc.
efed5e, 22 lines

```
// Down, Right, Up, Left
const int dx[4] = {1, 0, -1, 0};
const int dy[4] = {0, 1, 0, -1};
const char dc[4] = {'D', 'R', 'U', 'L'};

// Down, Down-Right, Right, Up-Right, Up, Up-Left, Left, Down-
   ↪Left
const int dx8[8] = {1, 1, 0, -1, -1, -1, 0, 1};
const int dy8[8] = {0, 1, 1, 1, 0, -1, -1, -1};

// Diagonals only
const int dxDiag[4] = {1, 1, -1, -1};
const int dyDiag[4] = {1, -1, 1, -1};

// Knight moves
const int dxK[8] = {2, 2, 1, 1, -1, -1, -2, -2};
const int dyK[8] = {1, -1, 2, -2, 2, -2, 1, -1};

// 3D Grid
const int dx[6] = {1, -1, 0, 0, 0, 0};
const int dy[6] = {0, 0, 1, -1, 0, 0};
const int dz[6] = {0, 0, 0, 0, 1, -1};
```